

Chapter 1: Introduction

Modular, Updateable Filesystem for Bare Metal as a Service (ModFS)

1 Context and Motivation

In modern cloud infrastructure, edge environments, and Bare Metal as a Service (BAAS) platforms, rapid, predictable, and storage-efficient server provisioning is a core operational requirement. BAAS platforms automate the lifecycle of physical servers, provisioning raw hardware nodes with operating system images on demand. Unlike virtualized multi-tenant environments where hypervisors or container runtimes isolate workloads on shared hosts, bare-metal deployment requires flashing full root filesystem images directly onto host storage media or serving them over network-attached block devices.

Historically, BAAS image management has relied on monolithic, whole-disk binary blobs. In this paradigm, every operating system configuration, application stack, or tenant environment is built, compressed, and stored as an independent, opaque disk image. When multiple server workloads share a common operating system baseline—such as an Ubuntu 22.04 LTS (Jammy Jellyfish) installation running on the x86-64 architecture with APT/dpkg package management—this monolithic model introduces massive storage redundancy. For example, maintaining ten distinct server configurations (e.g., dedicated web servers, database nodes, build runners, and machine learning hosts) that all build upon the same underlying Ubuntu operating system baseline requires storing ten complete, redundant operating system copies on the provisioning server.

Furthermore, software updates and security patching in monolithic BAAS pipelines suffer from severe inefficiency. Updating a single low-level library (such as `libssl` or `glibc`) or a single application package requires completely rebuilding, re-indexing, and re-shipping every full disk image across the provisioning network. This whole-image operational pattern drives up server-side storage costs, increases network bandwidth consumption during node deployment, and introduces substantial deployment latency.

While containerization technologies (e.g., Docker) and virtual machine template pools mitigate redundancy in virtualized environments through image layering, bare-metal provisioning demands a native, high-performance filesystem layout that boots directly on host hardware under standard Linux `systemd` initialization while preserving fine-grained modularity and updateability.

2 Problem Statement

Existing image management solutions in Bare Metal as a Service lack modular granularity and fine-grained update mechanisms. Storing monolithic whole-disk images results in storage bloat that scales linearly with the number of provisioned use cases ($N \times B$, where B is the baseline image size and N is the number of workloads).

Naively attempting to compose modular filesystem deltas at runtime or build time by overlaying independently compiled package sets introduces severe filesystem-level and package-level state conflicts:

1. **Uncontrolled Dependency Drift and Version Skew:** Installing packages into independent module layers at different times against dynamic distribution repositories causes

package version skews, broken inter-module dependency chains, and implicit base operating system upgrades, destroying deployment predictability.

2. **File Path Collisions:** Independent module builds can overwrite overlapping file paths or place conflicting binaries in shared FHS (Filesystem Hierarchy Standard) paths (e.g., `/usr/bin`, `/etc`), leading to file loss or layer-order-dependent binary execution.
3. **State Divergence in Shared Registries:** Multiple package installations modify shared, unowned system registries—global system databases that record installation state, user accounts, and system configuration. Key examples include:
 - The package status database (`/var/lib/dpkg/status`).
 - Configuration database records (`debconf config.dat` and `templates.dat`).
 - System account databases (`/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/gshadow`, `/etc/subuid`, `/etc/subgid`).
 - The alternative symlink registry (`/var/lib/dpkg/alternatives/*` and `/etc/alternatives/*`).
 - The dynamic linker binary cache (`/etc/ld.so.cache`).
 - The package diversion database (`/var/lib/dpkg/diversions`).
 - Extended package auto-installation states (`/var/lib/apt/extended_states`).

Overlaying these files naively using standard union filesystems causes the topmost layer to overwrite lower layers, rendering lower-layer packages invisible to package management tools or breaking system services.

4. **Numeric Identity Collisions:** Independent maintainer scripts allocate numeric User IDs (UIDs) and Group IDs (GIDs) dynamically at build time (e.g., `adduser -system`). Two independent modules can allocate the exact same numeric UID/GID (e.g., UID 103) to different service account names (e.g., `redis` vs. `memcache`), causing severe security and permission corruption when stacked.
5. **Directory Erasure via Opaque Markers:** Standard union filesystems (OverlayFS) write extended attributes (`trusted.overlay.opaque`) when directories are recreated during package builds. Stacking deltas containing these opaque markers causes upper layers to silently erase directory contents originating from lower layers without generating a direct file path collision.
6. **Runtime Resource Races:** Even when package sets compose cleanly at the filesystem layer without metadata conflicts, runtime services can collide over unmanaged physical hardware resources (such as two web servers, `nginx` and `apache2`, competing to bind to port 80).

3 Strategy and Proposed Solution: ModFS

To solve these challenges, this thesis presents **ModFS**, a modular, updateable filesystem framework engineered specifically for Bare Metal as a Service. ModFS replaces opaque whole-disk binary blobs with thin, composable module deltas built upon a shared, immutable base operating system image.

The core design of ModFS rests on four key architectural principles:

1. **Archive Pinning:** Every build in the system draws from a single, fixed archive repository snapshot (`snapshot.ubuntu.com/ubuntu/<TIMESTAMP>`), freezing all package versions and dependency resolution to an exact point in time.

2. **Base and Pinned Sibling Deltas:** A single base system image is constructed using `debootstrap` and fully upgraded against the pinned snapshot. Thin per-use-case modules are created by mounting the parent base read-only as a lower layer in OverlayFS, installing target packages into the merged view, and squashing *only* the upper directory (`upperdir`) into a compressed SquashFS archive (`.sqsh`).
3. **Semantic State Reconciliation:** To compose a bootable image, sibling modules are stacked as read-only lower layers alongside a dynamically computed **reconciliation layer** placed on top. This reconciler executes record-level union functions over diverged system registries (`dpkg status`, `debconf`, `passwd`, `group`, `shadow`, `gshadow`, `subuid`, `subgid`, `extended_states`, `diversions`) and regenerates tool-owned derived state (`ld.so.cache`, `/etc/alternatives/*`).
4. **Multi-Tier Evaluation Framework:** To ensure that a composed set of modules forms a valid, bootable system prior to bare-metal node flashing, ModFS implements a three-tier validation pipeline:
 - **Tier 1 (Static Metadata Admission):** Fast, rootless evaluation over declarative module manifests (`module.json`) to verify dependency closure and detect conflict classes.
 - **Tier 2 (Structural & Registry Verification):** Physical layer composition and verification of exact registry record unions, symlink target integrity, and file visibility across stacked layers.
 - **Tier 3 (QEMU UEFI Boot Testing):** Automated virtual machine boots under UEFI and `systemd` to verify service startup and detect runtime resource collisions.

4 Central Claim and Research Questions

The **central thesis claim** of this work is that **archive snapshot pinning reduces an otherwise intractable file-level conflict problem into a small, declarative, package-level verification problem**.

To systematically validate this central claim, this thesis investigates three primary **Research Questions (RQs)**:

- **RQ1 (Conflict Taxonomy & Reconciliation):** How can file-level and package-level state conflicts across independent filesystem deltas be systematically classified, prevented, and reconciled?
- **RQ2 (Storage Efficiency):** What server-side storage savings does the pinned sibling delta model achieve compared to traditional monolithic disk images across diverse workload scales?
- **RQ3 (Verification & Boot Reliability):** How effectively can static metadata admission and structural verification predict the bootability and runtime service health of composed modular systems?

5 Scope and Constraints

This thesis evaluates ModFS under the following specific parameters and engineering constraints:

- **Target Operating Environment:** Ubuntu 22.04 LTS (Jammy Jellyfish), APT/`dpkg` package management, and x86-64 processor architecture.

- **Module Topology:** All modules exist as direct **pinned siblings** on top of the shared base image; delta-on-delta nesting is explicitly out of scope.
- **Deployment Model:** Target bare-metal nodes are fully flashed with a flattened root filesystem image (`ext4`) constructed on the provisioning server; ModFS does not execute on-node runtime module switching or live rollback.
- **Update Lifecycle:** Moving the archive snapshot pin rebuilds the base and all sibling modules atomically per generation move.

6 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Fundamentals & Related Work** reviews Linux storage and union filesystem primitives (OverlayFS, SquashFS), `dpkg/APT` package mechanics, SAT-based package solvers, functional deployment models (Nix/NixOS), reproducible build standards, and distributed caching filesystems (AFS, Coda), factually positioning ModFS against 15 key literature sources.
- **Chapter 3: Architecture & Implementation** details the 17-stage ModFS pipeline, the pinned sibling delta builder, the 9-class conflict taxonomy, UID/GID range partitioning (ranges 2000+), and the record-level semantic reconciliation algorithms.
- **Chapter 4: Evaluation & Results** presents empirical results across the 40-module evaluation catalogue, analyzing static Tier 1 admission timings, Tier 2 composition scaling fits ($175.2 \text{ ms} + 30.55 \text{ ms} \times N$), storage reduction ratios ($5.59\times$ small cohort, $1.20\times$ large cohort, $1.84\times$ whole catalogue), and QEMU UEFI boot causal matrices.
- **Chapter 5: Discussion & Limitations** evaluates build non-determinism, atomic pin update costs, fat-base trade-offs, threats to validity, and identified system limits.
- **Chapter 6: Conclusion & Future Work** summarizes core contributions and outlines open research directions.
- **Appendix:** Contains reference provenance tables, data reconciliation details, the mandatory LUH AI tool disclosure table (*Verwendete Hilfsmittel*), and the signed Declaration of Independence (*Eigenständigkeitserklärung*).

Chapter 2: Fundamentals and Related Work

Comprehensive Theoretical Foundations and State-of-the-Art Positioning

ModFS Thesis Draft

September 21, 2026

Contents

1	Introduction and Overview	3
2	System Background and Linux Primitives	3
2.1	Package Management Mechanics and Maintainer Scripts	3
2.2	The Seven Unowned System Registries	4
2.2.1	1. Package Status Database (/var/lib/dpkg/status)	4
2.2.2	2. Configuration Database (debconf / config.dat, templates.dat)	4
2.2.3	3. User and Group Account Databases (/etc/passwd, /etc/group, /etc/shadow, /etc/gshadow, /etc/subuid, /etc/subgid)	5
2.2.4	4. Extended Package States (/var/lib/apt/extended_states)	5
2.2.5	5. Alternative Symlink Registry (/var/lib/dpkg/alternatives/* and /etc/alternatives/*)	5
2.2.6	6. File Diversion Registry (/var/lib/dpkg/diversions)	5
2.2.7	7. Dynamic Linker Cache (/etc/ld.so.cache)	6
2.3	OverlayFS Mechanics and Mutation Capture	6
2.4	Immutable Compressed Storage via SquashFS	6
3	Namespace Unification and Layered Filesystems	7
3.1	Foundational Stackable and Union Filesystems	7
3.1.1	4.BSD Union Mounts (Pendry & McKusick 1995)	7
3.1.2	Unionfs (Wright et al. 2006)	7
3.1.3	Plan 9 Name Spaces (Pike et al. 1993, 1995)	8
3.2	Comparative Positioning of ModFS Layer Stacking	8
4	Formal Package Co-Installability and Dependency Solvers	9
4.1	Declarative Repository Models and SAT Encodings	9
4.1.1	NP-Completeness and SAT Solvers	9
4.1.2	Certified Co-installability Kernels (coinst)	9
4.2	The Critical Gap: Metadata Solvers vs. Physical File System State	9
4.3	Positioning the ModFS Conflict Taxonomy and Reconciliation	10
5	Deterministic Builds, Functional OS Models, and Image Management	10
5.1	Purely Functional Deployment: Nix and NixOS	10
5.2	Reproducible Builds and Build Environment Control	10
5.3	Positioning ModFS Determinism and Integrity Seals	11

6	Distributed and Caching Storage Architectures	11
6.1	Whole-File Caching and Open/Close Consistency: AFS	12
6.2	Optimistic Disconnected Operation and Mutation Replay: Coda	13
6.3	Positioning ModFS Multi-Tier Verification (M5)	13
7	Summary of Prior Art Positioning and Novel Contributions	13

1 Introduction and Overview

This chapter establishes the comprehensive scientific foundations, operating system primitives, and related research literature that underpin **ModFS**: a modular, updateable filesystem architecture engineered specifically for Bare Metal as a Service (BAAS) infrastructure. ModFS replaces monolithic whole-disk image distribution with thin, composable, compressed filesystem deltas built upon a shared base image and reconciled at the record level.

To rigorously situate ModFS within the state of the art, this chapter analyzes five distinct domain areas across fifteen foundational reference papers:

1. **System Background and Linux Primitives (§2)**: Package management mechanics (`dpkg`/`APT`), maintaining unowned system registries, OverlayFS union mounting semantics, and SquashFS block compression.
2. **Namespace Unification and Layered Filesystems (§3)**: Early stackable filesystems (4.4BSD union mounts, `Unionfs`) and Plan 9 per-process name spaces, contrasting dynamic VFS lookup interception with ModFS’s pre-computed static SquashFS delta squashing.
3. **Formal Package Co-Installability and Dependency Solvers (§4)**: SAT/CP-based repository analysis (`EDOS`, `Mancoosi`, `coinst`) and the critical gap between declarative meta-data graphs and physical filesystem state divergence.
4. **Deterministic Builds, Functional OS Models, and Image Management (§5)**: Purely functional store paradigms (`Nix`/`NixOS`), reproducible build standards (`SOURCE_DATE_EPOCH`), and archive snapshot pinning.
5. **Distributed and Caching Storage Architectures (§6)**: Client-side disk caching (`AFS`) and optimistic disconnected reintegration (`Coda`), juxtaposed against ModFS’s server-side image verification pipeline.

Throughout this chapter, ten specific architectural mechanisms of ModFS (designated **M1** through **M10**) are systematically evaluated against prior art using exact source quotations and factual comparisons.

2 System Background and Linux Primitives

Modern Linux distributions provide software modularity through package management systems and kernel-level filesystem abstractions. ModFS targets the **Ubuntu 22.04 LTS (Jammy Jellyfish)** distribution operating on the `x86-64` architecture under the standard `APT` and `dpkg` software management toolchain.

2.1 Package Management Mechanics and Maintainer Scripts

In Debian-derived Linux distributions, software lifecycle management is split between low-level archive execution (`dpkg`) and high-level dependency resolution (`APT`). Packages are delivered as compressed binary archives (`.deb`) containing compiled executables, shared libraries, configuration files, control metadata, and executable shell scripts known as **maintainer scripts** (`preinst`, `postinst`, `prerm`, `postrm`).

When `APT` installs a set of requested packages, the installation proceeds through four distinct operational phases:

1. **Dependency Resolution and Order Calculation**: `APT` parses repository `Packages` indices, resolves disjunctive dependencies, and computes a topologically sorted execution plan.

2. **Archive Extraction and Direct File Unpacking:** `dpkg` extracts the payload tarball of each package directly into the target filesystem root (/). Every unpacked file is registered in the package's ownership manifest (`/var/lib/dpkg/info/<package>.list`).
3. **Maintainer Script Execution:** `dpkg` executes maintainer scripts (`preinst` before unpacking payload, `postinst` after unpacking payload). Maintainer scripts perform arbitrary imperative operations, such as creating system users via `adduser`, registering configuration templates via `debconf`, updating symlinks via `update-alternatives`, and populating database schemas.
4. **Trigger Processing and Deferred Hooks:** `dpkg` runs registered trigger routines to update global caches (e.g., invoking `ldconfig` to rebuild shared library search structures or `update-initramfs` to build boot ramdisks).

2.2 The Seven Unowned System Registries

A fundamental challenge in modular filesystem composition is that package installations modify two distinct categories of state: *owned files* and *unowned system registries*. Owned files belong explicitly to a single package (e.g., `/usr/bin/nginx`) and are tracked in its `.list` manifest. Unowned system registries, in contrast, are global database files that are not owned by any individual package but are mutated dynamically by multiple packages during maintainer script execution.

ModFS identifies seven critical system registries that inevitably diverge when packages are installed into separate filesystem deltas:

2.2.1 1. Package Status Database (`/var/lib/dpkg/status`)

The `dpkg` status database is a plain-text file formatted as a sequence of RFC-822-style stanzas. Each stanza records the authoritative state of a single package on the host:

```
Package: nginx-core
Status: install ok installed
Priority: optional
Section: httpd
Installed-Size: 1478
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Version: 1.18.0-6ubuntu14.4
Depends: libc6 (>= 2.34), libpcre3, zlib1g (>= 1:1.1.4)
Provides: httpd, httpd-cgi
Description: nginx core binary...
```

When two independent module deltas install packages on top of the same base system, each delta modifies its local `/var/lib/dpkg/status` file. Stacking these deltas in a standard union filesystem causes the upper layer's `status` file to completely overwrite the lower layer's `status` file. As a result, all packages installed in the lower layer become invisible to `dpkg`, rendering the system unmaintainable and breaking dependency checking.

2.2.2 2. Configuration Database (`debconf` / `config.dat`, `templates.dat`)

The Debian configuration management system (`debconf`) maintains two binary/text backend files under `/var/cache/debconf/`: `config.dat` (which stores user configuration selections, question flags, and variable bindings) and `templates.dat` (which defines the schema and internationalized prompts for package queries). Maintainer scripts query and update `debconf` during

installation via the `debconf-communicate` protocol. Independent package installations alter these databases, causing last-writer-wins overwrite conflicts when stacked.

2.2.3 3. User and Group Account Databases (`/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/gshadow`, `/etc/subuid`, `/etc/subgid`)

POSIX identity management relies on flat text databases mapping account names to numeric identifiers (UIDs and GIDs):

- `/etc/passwd`: Maps usernames to numeric UIDs, primary GIDs, home directories, and default shells.
- `/etc/group`: Maps group names to numeric GIDs and user membership lists.
- `/etc/shadow` & `/etc/gshadow`: Store encrypted password hashes, expiration dates, and administrative flags.
- `/etc/subuid` & `/etc/subgid`: Subordinate UID/GID ranges allocated for unprivileged user namespace container execution.

Package maintainer scripts frequently create system accounts dynamically using commands such as `adduser -system -group -home /var/lib/redis redis`. When system accounts are allocated dynamically across independent module builds without coordination, two independent modules can allocate the exact same numeric UID (e.g., UID 105) to entirely different service accounts (e.g., `redis` in Module A vs. `postgres` in Module B), leading to critical security breaches and file permission corruption.

2.2.4 4. Extended Package States (`/var/lib/apt/extended_states`)

APT maintains an explicit record of whether each installed package was manually requested by the system administrator or automatically pulled in as a transitive dependency:

```
Package: libpcre3
Auto-Installed: 1
```

```
Package: nginx
Auto-Installed: 0
```

This state governs automatic package pruning (`apt-get autoremove`). Independent module installations append stanzas to `extended_states`, requiring record-level merging during composition.

2.2.5 5. Alternative Symlink Registry (`/var/lib/dpkg/alternatives/*` and `/etc/alternatives/*`)

Debian manages default command selection (e.g., choosing which binary fulfills `/usr/bin/editor` or `/usr/bin/java`) through `update-alternatives`. The authority database under `/var/lib/dpkg/alternatives/` records priority scores, slave links, and available provider paths. The corresponding symlinks in `/etc/alternatives/` point to the highest-priority registered provider. Independent package installations append alternative choices, requiring both database record merging and dynamic symlink recalculation.

2.2.6 6. File Diversion Registry (`/var/lib/dpkg/diversions`)

When a package must overwrite a file belonging to another package, `dpkg-divert` instructs `dpkg` to write the original file to a diverted location (e.g., `/usr/bin/gcc.real`). The registry at `/var/lib/dpkg/diversions` tracks all active redirections. Divergent diversions across modules must be unified to prevent package file collisions.

2.2.7 7. Dynamic Linker Cache (/etc/ld.so.cache)

The ELF dynamic loader (`ld.so`) resolves shared object dependencies (`.so` files) at runtime using an optimized binary cache file (`/etc/ld.so.cache`). This file is generated by `ldconfig`, which scans standard library paths (`/lib`, `/usr/lib`, `/usr/local/lib`) and candidate configuration files under `/etc/ld.so.conf.d/`. Because `/etc/ld.so.cache` is an unowned binary artifact, overlaying deltas invalidates the cache unless it is regenerated across the unified layer view.

2.3 OverlayFS Mechanics and Mutation Capture

Linux **OverlayFS** is an in-kernel union filesystem that presents a single merged directory tree derived from multiple underlying directories (branches). An OverlayFS mount requires four physical directory paths:

`mount -t overlay overlay -o lowerdir =  $L_1 : L_2 : \dots : L_n$ , upperdir =  $U$ , workdir =  $W$   $M$  (1)`

where $L_1 \dots L_n$ are read-only lower directories, U is a read-write upper directory, W is a dedicated work directory on the same filesystem as U , and M is the target merged mount point.

OverlayFS intercepts Virtual Filesystem (VFS) operations and handles state modifications according to three primary semantics:

- **Copy-On-Write (COW)**: When an application opens a lower-layer file for writing, OverlayFS copies the complete file from the read-only `lowerdir` to the writable `upperdir` before executing the modification. File permissions, timestamps, owner UIDs, and extended attributes are preserved during copy-up.
- **Whiteouts** (`c 0 0`): When an application deletes a file originating from a read-only lower layer, OverlayFS creates a special `whiteout` character device node in `upperdir` with major device number 0 and minor device number 0. During path lookup (`lookup()`) and directory enumeration (`readdir()`), the kernel recognizes whiteout nodes and hides the corresponding lower-layer file from user space.
- **Opaque Directories** (`trusted.overlay.opaque = "y"`): When a directory existing in a read-only lower layer is deleted and subsequently re-created in the merged view (e.g., executing `rm -rf /etc/conf.d && mkdir /etc/conf.d`), OverlayFS creates the new directory in `upperdir` and attaches an extended attribute `trusted.overlay.opaque = "y"`. When looking up paths inside an opaque directory, the OverlayFS kernel driver stops scanning any lower layers, preventing lower-layer directory contents from appearing inside the newly created directory.

2.4 Immutable Compressed Storage via SquashFS

SquashFS is a read-only, block-compressed filesystem driver built into the Linux kernel. SquashFS packages entire directory hierarchies, regular files, symbolic links, character/block device nodes, FIFOs, sockets, and extended attributes into a dense, contiguous binary image (`.sqsh`).

ModFS utilizes SquashFS to encapsulate OverlayFS upper directories into portable, read-only module archives. SquashFS is uniquely suited for delta storage due to three technical characteristics:

1. **Device Node and Xattr Preservation**: SquashFS natively preserves character device nodes—including major/minor 0:0 OverlayFS whiteout devices—and extended attributes (such as `trusted.overlay.opaque`).
2. **High Compression Ratios**: Utilizing modern block compression algorithms (`zstd` with 128 KB or 1 MB block sizes), SquashFS achieves 60% to 75% storage compression compared to raw uncompressed tarballs or ext4 images.

3. **Direct Kernel Mountability:** SquashFS archives can be mounted directly by the Linux kernel via loop devices without requiring prior decompression or extraction to temporary disk storage.

3 Namespace Unification and Layered Filesystems

The concept of unifying independent directory trees into a single, cohesive namespace has a rich history in operating system research. This section examines foundational stackable filesystems and contrasts their runtime lookup interception models against ModFS’s static delta squashing architecture.

3.1 Foundational Stackable and Union Filesystems

3.1.1 4.4BSD Union Mounts (Pendry & McKusick 1995)

Pendry and McKusick [1] introduced in-kernel union mounts in 4.4BSD-Lite. Their design allows directory hierarchies to be mounted transparently over existing directories using VFS vnode stacking.

“Whenever a whiteout is replaced by a directory, the directory inode is automatically tagged with the opaque attribute... During a lookup in a union stack, the union filesystem does not continue to the lower layer object if the upper layer object is marked opaque. To understand this requirement, consider the sequence of operations: `rm -rf tree`, `mkdir tree`... By marking the newly created upper layer directory with the opaque attribute the union filesystem ensures that the semantics prevents the lower layer objects in tree from becoming visible.” — Pendry & McKusick [1], Section 5.1

The 4.4BSD union mount mechanism established two core primitives used in modern union filesystems:

- **Layered Priority:** Lookup requests scan layers from top to bottom, returning the first matching file name.
- **Opaque Directories and Whiteouts:** File deletion in read-only lower layers is recorded by creating explicit whiteout directory entries (`DT_WHT`) in the upper layer, while newly created directories that replace deleted trees are marked with an `opaque` flag to stop lower-layer lookup continuation.

3.1.2 Unionfs (Wright et al. 2006)

Wright et al. [2] presented Unionfs, an n -way fan-out stackable filesystem for Linux operating directly over arbitrary VFS physical branches.

“Unionfs is a stackable file system that operates on multiple underlying file systems. It has an n -way fan-out architecture... In this example, the branches are a read-write Ext2 file system, a read-only Ext2 file system, a read-write NFS file system, and a read-only ISO9660 file system.” — Wright et al. [2], Section 2

Unionfs introduced dynamic branch management (allowing branches to be inserted, removed, or re-ordered at runtime) and complex copy-up handling across mixed read-write and read-only storage media. However, Unionfs operates strictly at runtime via live VFS call interception: all lookups, directory enumerations (`readdir`), and duplicate suppression algorithms run inside the kernel during active file system access.

3.1.3 Plan 9 Name Spaces (Pike et al. 1993, 1995)

Pike et al. [3, 4] developed per-process customizable filesystem name spaces in the Plan 9 operating system. Plan 9 allows processes to assemble union directories using `bind` and `mount` system calls with flags such as `MAFTER` or `MBEFORE`.

Unlike Unix union filesystems that perform copy-on-write file modifications, Plan 9 union directories operate strictly as simple ordered lists of directories evaluated during path traversal. File creation always occurs in the first directory of the union stack that permits writes, while file deletion removes only the first visible instance of a file without creating whiteout markers.

3.2 Comparative Positioning of ModFS Layer Stacking

Table 1 factually positions ModFS’s delta stacking mechanisms (**M1**, **M9**, **M10**) against prior stackable filesystem literature.

Table 1: Factual Positioning of ModFS Layer Mechanisms Against Prior Art

Mechanism	Prior Art Source	Exact Source Quotation	Specific Difference
M1: Delta Squashing	Wright et al. (2006) [2], Pendry & McKusick (1995) [1]	<i>“Unionfs is a stackable file system that operates on multiple underlying file systems...”</i> [2]	Unionfs and 4.4BSD union mounts dynamically layer live filesystem branches at runtime and execute copy-up on write, whereas ModFS squashes isolated build-time OverlayFS upperdirs into immutable, compressed SquashFS delta archives ahead of time and composes them statically.
M9: Order-Independent Composition	<i>None in 15 sources</i>	<i>No overlap found in the 15 sources.</i>	Standard union filesystems resolve lookup collisions by strict top-to-bottom layer priority (last-writer-wins), whereas ModFS achieves order-independent composition for packages by merging diverged system registries at record granularity into a top reconciliation layer.
M10: Opaque Marker Stripping	Pendry & McKusick (1995) [1]	<i>“Whenever a whiteout is replaced by a directory, the directory inode is automatically tagged with the opaque attribute...”</i> [1]	Pendry & McKusick establish opaque attributes to prevent lower-layer files from shining through after directory re-creation, whereas ModFS identifies that build-time opaque markers (<code>trusted.overlay.opaque</code>) cause inter-module directory erasure at compose time (Class 9 conflict) and explicitly strips opaque attributes from module deltas prior to squashing.

4 Formal Package Co-Installability and Dependency Solvers

A central domain of research in software engineering is formal package repository analysis. This section reviews declarative dependency solvers and contrasts their metadata models with physical filesystem state reconciliation.

4.1 Declarative Repository Models and SAT Encodings

The EDOS (Evolution and Quality of Open Source Software) and Mancoosi projects formalized package repository co-installability using mathematical logic [5, 6, 7].

A package repository is formally defined as a tuple:

$$R = (P, D, C) \quad (2)$$

where P is a finite set of package identifiers, $D : P \rightarrow \mathcal{P}(\mathcal{P}(P))$ is a dependency function mapping each package to a set of dependency disjunctions, and $C \subseteq P \times P$ is a symmetric, irreflexive declared conflict relation.

4.1.1 NP-Completeness and SAT Solvers

Mancinelli et al. [6] and Treinen & Zacchiroli [5] proved that determining whether a set of packages can be co-installed under repository dependencies D and conflicts C is NP-complete by reduction from 3-SAT.

To verify repository consistency, tools like `edos-debcheck` convert package metadata into Conjunctive Normal Form (CNF) boolean formulas:

- If package A depends on $B \vee C$, it generates the clause: $\neg A \vee B \vee C$.
- If package A conflicts with B , it generates the clause: $\neg A \vee \neg B$.

High-performance SAT solvers (such as `pkglab`) then evaluate whether a satisfying truth assignment exists for target package installations.

4.1.2 Certified Co-installability Kernels (`coinst`)

Vouillon and Di Cosmo [7] developed `coinst`, a formally certified graph reduction framework that extracts minimal co-installability kernels from vast package repositories.

“A repository is a tuple $R = (P, D, C)$ where P is a finite set of packages, $D : P \rightarrow \mathcal{P}(\mathcal{P}(P))$ is the dependency function... and C , a symmetric irreflexive relation over P , is the conflict relation.” — Vouillon & Di Cosmo [7], Section 2.1

4.2 The Critical Gap: Metadata Solvers vs. Physical File System State

While SAT-based metadata solvers rigorously verify declarative package dependencies (D) and explicit conflict declarations (C), they possess a critical limitation: **they operate entirely outside physical filesystem reality**.

As Vouillon & Di Cosmo [7] explicitly state in their formal model, packages are treated as opaque atomic nodes $p \in P$. The formal model contains no representation of on-disk file paths, directory structures, maintaining script execution, or system registries.

Treinen & Zacchiroli [5] recognized this limitation and introduced a static file collision check:

“A first idea towards a better solution is to only consider those pairs of packages that actually share at least one file. Luckily, the information which package contains which file is available in the file Contents of the distribution... Co-installability is checked with the `pkglab` tool... This first phase gives us a reduced list of pairs of packages.” — Treinen & Zacchiroli [5], Section 2.2.5

However, intersecting the distribution `Contents` index only detects shared *owned* file collisions. It cannot detect:

1. **State Divergence in Unowned Registries:** Two packages that do not share any owned files will both modify `/var/lib/dpkg/status` or `/etc/passwd`, corrupting system state when overlaid.
2. **Numeric Identity Collisions:** Independent maintainer scripts allocating identical UIDs to different account names.
3. **Opaque Directory Erasure:** Build-time `trusted.overlay.opaque` extended attributes erasing lower-layer files.
4. **Runtime Service Races:** Independent modules attempting to bind the same network socket (e.g., TCP port 80).

4.3 Positioning the ModFS Conflict Taxonomy and Reconciliation

Table 2 positions ModFS’s reconciliation mechanisms (**M2**, **M3**, **M4**) against declarative package solver literature.

5 Deterministic Builds, Functional OS Models, and Image Management

Achieving predictable software deployment requires isolating package build environments and eliminating non-deterministic side effects. This section evaluates functional deployment models and reproducible build techniques.

5.1 Purely Functional Deployment: Nix and NixOS

Dolstra et al. [8, 9] introduced the purely functional software deployment model in the Nix package manager and NixOS distribution.

“Directory names in the store are chosen so as to uniquely identify revisions and variants of components... computed by hashing all inputs involved in building the component.”
— Dolstra et al. [8], Section “Nix Store”

Nix stores all software components under unique, cryptographic hash-addressed directory paths inside the central `/nix/store` directory (e.g., `/nix/store/8dpf...-gcc-12.2.0`). Nix achieves side-by-side co-existence of conflicting package versions by abandoning the standard Filesystem Hierarchy Standard (FHS). Global directories such as `/bin`, `/lib`, and `/usr` are eliminated; binaries locate dependencies using hardcoded ELF `RPATH` header fields pointing directly to specific store paths.

In NixOS [9], the entire operating system configuration is compiled declaratively from pure functional expressions into a tree of symlinks managed under `/nix/store`.

5.2 Reproducible Builds and Build Environment Control

Lamb and Zacchiroli [10] formalize the Reproducible Builds architecture, which aims to ensure that identical source code and build tools produce bit-for-bit identical binary packages (`.deb`).

“The build process of a software product is reproducible if, after designating a specific version of its source code and all of its build dependencies, every build produces bit-for-bit identical artifacts, no matter the environment in which the build is performed.”
— Lamb & Zacchiroli [10], Section “Reproducible Builds”, Def. 1

Table 2: Factual Positioning of ModFS Reconciliation Mechanisms Against Prior Art

Mechanism	Prior Art Source	Exact Source Quotation	Specific Difference
M2: Record Registry Reconciliation	<i>None in 15 sources</i>	<i>No overlap found in the 15 sources.</i>	Package managers overwrite registry files wholesale or rely on live maintainer scripts, whereas ModFS executes semantic record-level union functions over non-package-owned, diverged registries (<code>dpkg status</code> , <code>debconf</code> , <code>passwd</code> , <code>group</code> , <code>shadow</code> , <code>gshadow</code> , <code>subuid</code> , <code>subgid</code> , <code>diversions</code> , <code>extended_states</code>) inside a top reconciliation layer.
M3: Derived State Regeneration	<i>None in 15 sources</i>	<i>No overlap found in the 15 sources.</i>	Existing deployment systems attempt to merge or overwrite derived binary caches, whereas ModFS explicitly distinguishes registry record merging from derived state regeneration, recomputing <code>/etc/ld.so.cache</code> and <code>/etc/alternatives/*</code> symlinks using native owning tools (<code>ldconfig</code> , <code>update-alternatives -auto</code>) inside a merged chroot.
M4: 9-Class Conflict Taxonomy	Mancinelli et al. (2006) [6]	<i>“In this paper we have taken into account only the three main relationships that are used with the same semantics in both RPM and DEB packages: Depends... Conflicts... Pre-Depends...” [6]</i>	Mancinelli et al. categorize declared package metadata relations (<code>Depends</code> , <code>Conflicts</code>), whereas ModFS defines a 9-class taxonomy covering filesystem collisions, unowned registry divergence, numeric account ID overlap, build drift, and runtime service races.

Reproducible Builds identifies primary drivers of build non-determinism—including build timestamps, file ordering during directory traversal (`readdir()`), build path leaks, and CPU architecture flags—and establishes the `SOURCE_DATE_EPOCH` standard to freeze build-time clocks.

5.3 Positioning ModFS Determinism and Integrity Seals

Table 3 positions ModFS’s determinism mechanisms (**M6**, **M7**, **M8**) against Nix and Reproducible Builds prior art.

6 Distributed and Caching Storage Architectures

Deploying root filesystems across bare-metal server fleets relates conceptually to distributed storage architectures. This section examines client-side caching and disconnected operation in

Table 3: Factual Positioning of ModFS Determinism Mechanisms Against Prior Art

Mechanism	Prior Art Source	Exact Source Quotation	Specific Difference
M6: UID/GID Range Partitioning	<i>None in 15 sources</i>	<i>No overlap found in the 15 sources.</i>	Maintainer scripts allocate system numeric UIDs/GIDs dynamically at install time, causing identity collisions across deltas, whereas ModFS assigns an append-only, disjoint 100-wide UID/GID range (from 2000+) to each module manifest, configuring <code>adduser.conf</code> and <code>login.defs</code> during delta compilation to enforce isolation.
M7: Digest Integrity Seals	Lamb & Zacchiroli (2022) [10], Dolstra et al. (2004) [8]	<i>“Once a Debian package is built, the precise source version and the versions of all its build dependencies are recorded in a .buildinfo file...”</i> [10]	<code>.buildinfo</code> records build dependencies for audit logs and Nix hashes inputs into store paths, whereas ModFS embeds a canonical digest (<code>binding.fields_sha256</code>) over manifest fields directly inside the manifest to seal it to a pre-built SquashFS delta during admission.
M8: Snapshot Repository Pinning	Lamb & Zacchiroli (2022) [10], Dolstra & Löh (2008) [9]	<i>“The build process of a software product is reproducible if... every build produces bit-for-bit identical artifacts...”</i> [10]	Lamb & Zacchiroli focus on bit-for-bit compiler reproducibility and NixOS builds pure functional graphs, whereas ModFS uses archive snapshot pinning (<code>snapshot.ubuntu.com</code>) to freeze package dependency resolution while preserving standard FHS paths (<code>/usr</code> , <code>/etc</code> , <code>/var</code>).

distributed filesystems.

6.1 Whole-File Caching and Open/Close Consistency: AFS

Howard et al. [11] and Kazar [12] evaluated the Andrew File System (AFS). AFS achieves high scalability across large client fleets by caching entire files on local client storage.

AFS enforces **open/close consistency**: when a client opens a file (`open()`), it fetches the complete file from the Vice server and caches it locally. All subsequent read and write operations execute directly against the local disk cache. When the client closes the file (`close()`), any modified blocks are flushed back to the central server. State consistency across clients is maintained via server-initiated **callback invalidations**: when a file is modified, the server breaks callbacks on all other clients caching that file.

6.2 Optimistic Disconnected Operation and Mutation Replay: Coda

Satyanarayanan et al. [13] and Kistler & Satyanarayanan [14] extended AFS in the Coda File System to support optimistic disconnected operation.

During network partitions or disconnected operation, Coda clients (managed by the **Venus** cache manager) log all local VFS mutations (**create**, **write**, **unlink**, **mkdir**) in a persistent recoverability log (RVM). When network connectivity to Vice servers is restored, Coda executes automated reintegration by replaying the mutation log against the server and resolving directory conflicts via version vectors.

6.3 Positioning ModFS Multi-Tier Verification (M5)

Table 4 positions ModFS’s stratified verification pipeline (**M5**) against distributed network storage prior art.

Table 4: Factual Positioning of ModFS Verification Pipeline (M5) Against Prior Art

Mechanism	Prior Art Source	Exact Source Quotation	Specific Difference
M5: Stratified Multi-Tier Verification	<i>None in 15 sources</i>	<i>No overlap found in the 15 sources.</i>	Distributed filesystems (AFS, Coda) enforce runtime cache consistency or optimistic write log replay across client networks, whereas ModFS establishes a server-side 3-tier validation pipeline (Tier 1 rootless static admission, Tier 2 physical OverlayFS composition and registry record verification, Tier 3 automated QEMU UEFI boot testing) to verify modular root images prior to bare-metal node flashing.

7 Summary of Prior Art Positioning and Novel Contributions

This chapter has systematically evaluated the foundations of **ModFS** against fifteen reference papers from the operating systems, package management, functional build, and distributed storage literature.

The factual positioning across ModFS’s ten architectural mechanisms is summarized as follows:

1. **Non-Novel Mechanism Acknowledged:** ModFS explicitly acknowledges that OverlayFS opaque directory tagging (**trusted.overlay.opaque**) and whiteout handling draw directly from prior union mount foundations ([1, 2]), and that pair-wise file collision detection draws from static package index intersection techniques ([5]).
2. **Direct Architectural Divergence:** Unlike standard union filesystems ([1, 2]) that layer live branches at runtime and resolve collisions via top-to-bottom priority, ModFS squashes isolated upperdirs into compressed SquashFS deltas (**M1**) and reconciles diverged registries at record granularity to achieve order-independent package composition (**M9**).
3. **Addressing the Unowned State Gap:** While declarative SAT solvers ([6, 7]) operate exclusively on metadata graphs (P, D, C), ModFS bridges the gap to physical filesystem

state by executing semantic record-level union functions over seven unowned system registries (**M2**), regenerating derived dynamic linker caches (**M3**), and establishing a 9-class physical conflict taxonomy (**M4**).

4. **Deterministic Composition Under Standard FHS:** Unlike functional systems ([8, 9]) that eliminate global Linux paths (`/usr`, `/etc`), ModFS preserves standard FHS paths required by mainstream Debian/Ubuntu packages while achieving deterministic build resolution via archive snapshot repository pinning (**M8**), manifest integrity seals (**M7**), and append-only UID/GID range partitioning (**M6**).
5. **Stratified Pre-Flashing Admission:** Unlike network filesystems ([11, 13]) that enforce runtime cache invalidation, ModFS implements a 3-tier gating pipeline (**M5**) that catches metadata, structural, and UEFI boot failures on the server before bare-metal node flashing.

References

- [1] J.-S. Pendry and M. K. McKusick. Union mounts in 4.4BSD-Lite. In *Proceedings of the USENIX Technical Conference*, pages 25–33, 1995.
- [2] C. P. Wright, J. Dave, P. Zadok, and E. Zadok. Versatility and performance of stackable file systems. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.
- [3] R. Pike, D. Presotto, K. Thompson, and H. Trickey. The use of name spaces in Plan 9. *ACM Operating Systems Review*, 27(2):72–76, 1993.
- [4] R. Pike, D. Presotto, S. Dorward, B. Flandrena, K. Thompson, H. Trickey, and P. Winterbottom. Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254, 1995.
- [5] R. Treinen and S. Zacchiroli. Solving the package installability problem for realistic GNU/Linux distributions. *Software: Practice and Experience*, 38(14):1449–1472, 2008.
- [6] F. Mancinelli, J. Boender, R. Di Cosmo, J. Vouillon, B. Durak, X. Leroy, and R. Treinen. Managing the complexity of large free and open source package-based software distributions. In *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 199–208, 2006.
- [7] J. Vouillon and R. Di Cosmo. On co-installability essential kernels in software repositories. *Logical Methods in Computer Science*, 9(2), 2013.
- [8] E. Dolstra, M. de Jonge, and E. Visser. Nix: A safe and policy-free system for software deployment. In *Proceedings of the 18th Congress of the IFIP World Computer Congress*, pages 79–92, 2004.
- [9] E. Dolstra and A. Löb. NixOS: A purely functional Linux distribution. In *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 367–378, 2008.
- [10] C. Lamb and S. Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022.
- [11] J. H. Howard, M. L. Kazar, S. G. Menees, D. A. Nichols, M. Satyanarayanan, R. N. Sidebotham, and M. J. West. Scale and performance in a distributed file system. *ACM Transactions on Computer Systems (TOCS)*, 6(1):51–81, 1988.

- [12] M. L. Kazar. Synchronization and caching issues in the Andrew File System. In *Proceedings of the USENIX Winter Technical Conference*, pages 27–36, 1988.
- [13] M. Satyanarayanan, J. J. Kistler, P. Kumar, M. E. Okasaki, E. H. Siegel, and D. C. Steere. Coda: A highly available file system for a distributed workstation environment. *IEEE Transactions on Computers*, 39(4):447–459, 1990.
- [14] J. J. Kistler and M. Satyanarayanan. Disconnected operation in the Coda File System. *ACM Transactions on Computer Systems (TOCS)*, 10(1):3–25, 1992.
- [15] A. Uta, A. Custura, D. Epema, and A. Iosup. Is big data performance reproducible in the cloud? *IEEE Transactions on Parallel and Distributed Systems*, 31(12):2818–2833, 2020.

Chapter 3: Architecture and Implementation

ModFS Thesis

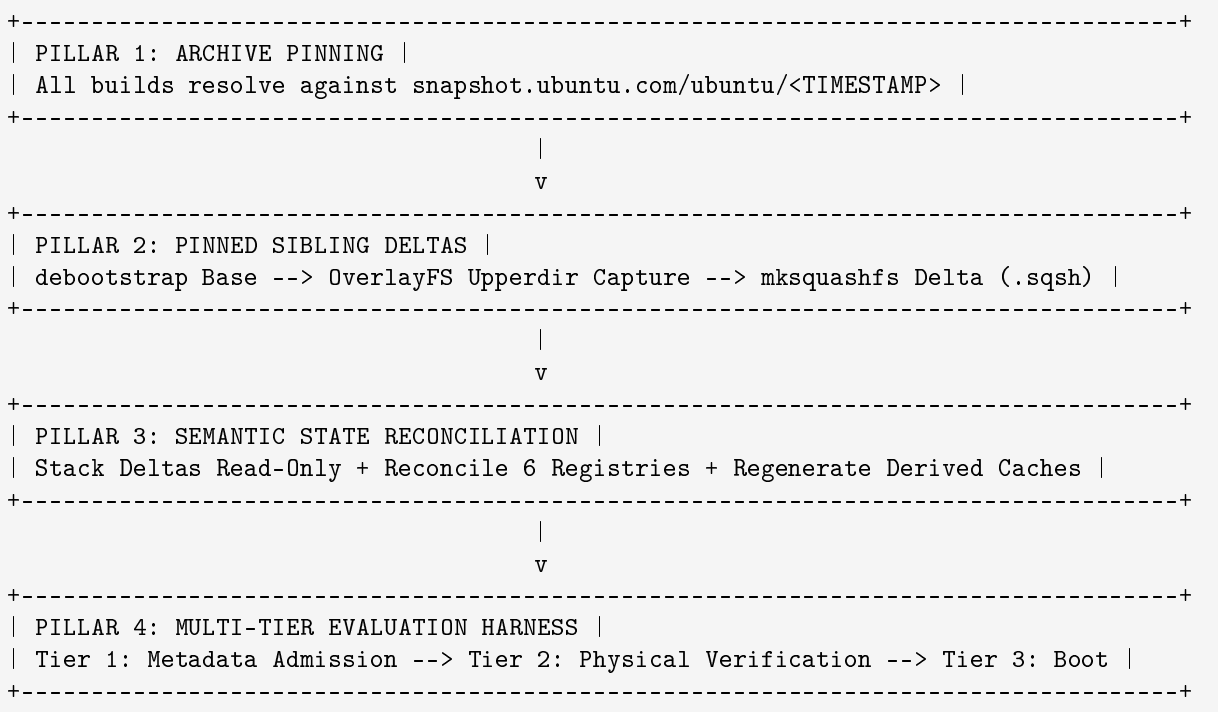
Contents

1	High-Level Architecture and Core Design Pillars	2
1.1	Architectural Constraints	3
2	Automated 17-Stage Build and Verification Pipeline	4
2.1	Stage-by-Stage Functional Breakdown	4
2.2	Deterministic Delta Build Mechanics	6
3	The 9-Class Conflict Taxonomy and Policy Engine	7
3.1	Detailed Breakdown of Conflict Classes	7
4	Append-Only UID/GID Range Partitioning	9
4.1	Prevention Architecture: Append-Only Allocation Windows	9
4.2	Account Audit and Database Reconciliation	10
5	Semantic Record-Level Registry Reconciliation	11
5.1	Category 1: Record-Level Merge Algorithms (<code>reconcile.py</code>)	11
5.2	Category 2: Derived State Regeneration	12
6	Module Manifest Schema, Dependency Closure, and Binding Seals	13
6.1	Manifest Schema Structure	13
6.2	Contribution Model and Dependency Reconstruction	14
6.3	Cryptographic Binding Seals	14
7	Catalogue Composition and Module Classification	15
7.1	Cohort Classification	15

1 High-Level Architecture and Core Design Pillars

ModFS is a modular, updateable filesystem architecture engineered specifically for Bare Metal as a Service (BAAS) infrastructure. Traditional BAAS image pipelines treat operating system environments as monolithic, opaque whole-disk binary blobs. When multiple distinct server workloads share a common operating system foundation (such as an Ubuntu LTS release), this monolithic model requires storing, versioning, and distributing full-disk images for every workload variant. For instance, provisioning ten distinct application environments that all build upon the same base operating system results in ten redundant full-disk copies on the server repository. Furthermore, updating or patching a single software package requires rebuilding, re-indexing, and re-shipping every full disk image across the network.

ModFS replaces opaque whole-disk blobs with thin, composable module deltas built upon a shared base operating system image. Rather than attempting to compose filesystems dynamically at runtime on host nodes or relying on ad-hoc package overlays, ModFS introduces a server-side build, verification, and composition pipeline that operates on four core architectural pillars:



1. **Archive Snapshot Pinning:** Every build in the system draws package repositories from a single, fixed archive snapshot (`snapshot.ubuntu.com/ubuntu/<TIMESTAMP>`). The timestamp identifier is explicitly embedded in the repository URL path. This freezes package version resolution across the entire distribution archive (`main`, `restricted`, `universe`, `multiverse`, `jammy-updates`, and `jammy-security`), ensuring that dependency solvers operate over a completely static package graph.
2. **Base and Pinned Sibling Deltas:** A single base system image (`base.sqsh`) is constructed using `debootstrap` and fully upgraded (`apt-get full-upgrade`) against the pinned archive snapshot. Thin per-use-case modules are created by mounting the base image read-only as a lower layer in OverlayFS, installing requested target packages into the merged view, and squashing *only* the upper directory (`upperdir`) into a compressed SquashFS archive (`.sqsh`). Because APT sees the base system as already installed, the `upperdir` captures only newly created or modified files. The filesystem captures the diff implicitly; ModFS never computes file diffs manually.

3. **Semantic State Reconciliation:** To compose a bootable OS image for a specific server workload, a set of module deltas is stacked as read-only lower layers alongside a dynamically computed **reconciliation layer** placed on top. This reconciler performs semantic record-level unions over diverged system registries that are rewritten by maintainer scripts but owned by no single package—including `dpkg` status stanzas, `debconf` configuration databases, user and group account records (`passwd`, `group`, `shadow`, `gshadow`, `subuid`, `subgid`), file diversions, and extended package states. Derived binary caches (such as `/etc/ld.so.cache` and `/etc/alternatives/*` symlinks) are recomputed inside the merged view using their native owning tools (`ldconfig`, `update-alternatives`).
4. **Multi-Tier Evaluation Framework:** Before flashing a composed image onto bare-metal hardware, ModFS executes a three-tier validation pipeline. Tier 1 performs rapid, rootless static metadata admission over module manifests to verify dependency closure and detect conflict classes. Tier 2 mounts physical SquashFS deltas, executes registry reconciliation, and verifies structural file visibility, symlink target validity, and registry completeness. Tier 3 packs a flattened UEFI image, boots it under QEMU, and executes automated service probes under `systemd` to detect runtime resource races.

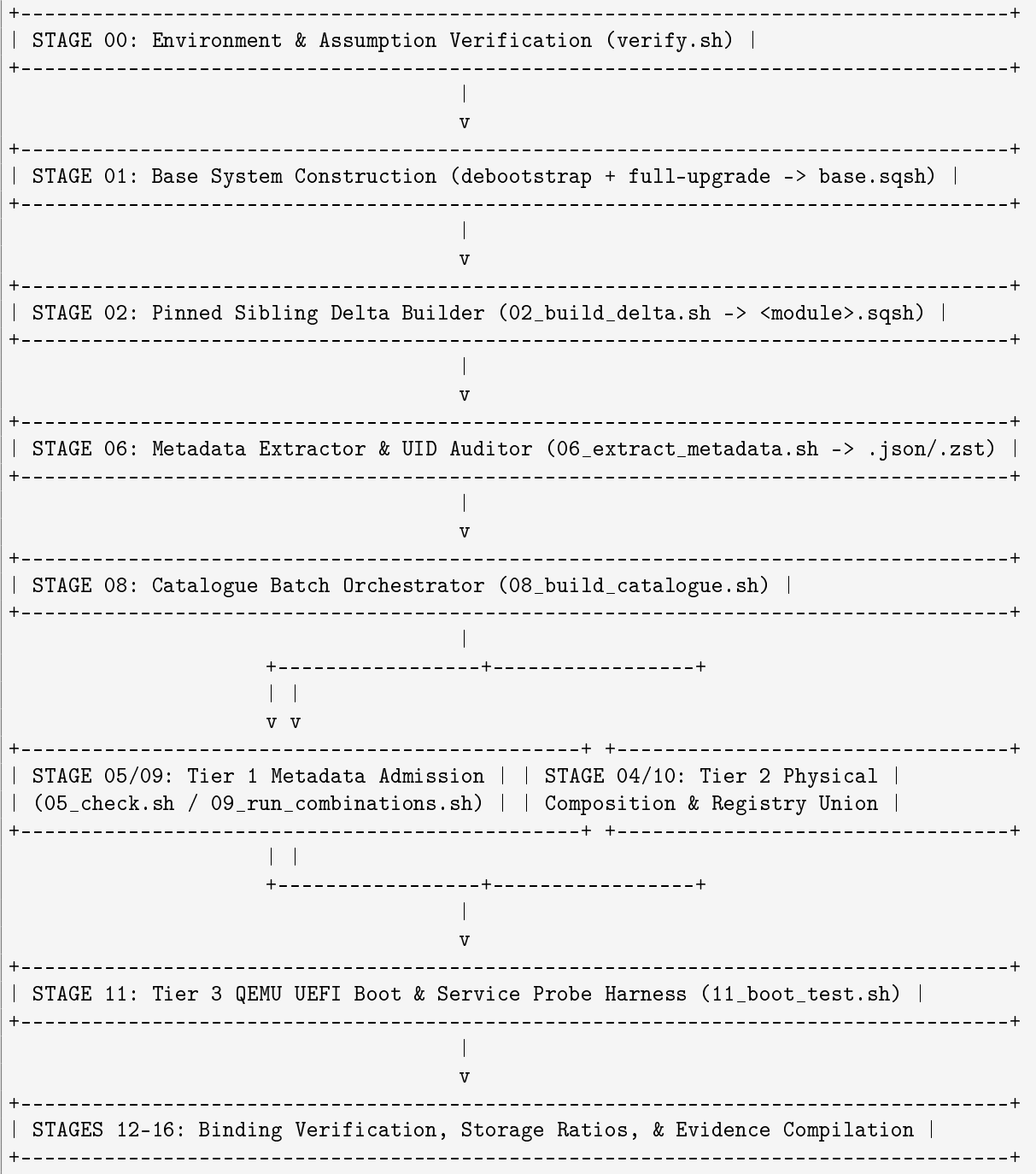
1.1 Architectural Constraints

To maintain strict determinism and prevent combinatorial state explosion, ModFS enforces explicit architectural boundaries:

- **Flat Sibling Topology:** All module deltas exist as direct **pinned siblings** built against the base image. Delta-on-delta nesting (building a module on top of another module) is strictly prohibited.
- **Atomic Generation Moves:** An archive snapshot timestamp defines an immutable **generation**. Moving the archive pin requires rebuilding the base image and all sibling modules atomically into a separate generation root (`MODFS_ROOT`). Mixing deltas across different generations or snapshots is explicitly forbidden.
- **Server-Side Image Flattening:** ModFS composes, reconciles, and flattens module deltas into an `ext4` root filesystem image on the build server. The provisioned bare-metal node receives a standard, flattened disk image. ModFS does not perform on-node runtime module switching or live rollback.

2 Automated 17-Stage Build and Verification Pipeline

The ModFS implementation is structured into an automated 17-stage pipeline (`00_verify.sh` through `16_build_evidence.sh`) supported by core Python reconciliation and verification utilities (`reconcile.py`, `verify_compose.py`).



2.1 Stage-by-Stage Functional Breakdown

1. **00_verify.sh (Environment Verification)**: Asserts kernel capability, required CLI utilities (`debootstrap`, `mksquashfs`, `zstd`, `qemu-system-x86_64`), archive snapshot reachability (`snapshot.ubuntu.com`), and OverlayFS whiteout/opaque attribute round-trip survival.
2. **01_build_base.sh (Base Image Construction)**: Executes `debootstrap` against the pinned

snapshot to build a minimal Ubuntu 22.04 LTS rootfs, chroots to execute `apt-get full-upgrade`, empties `/etc/machine-id`, strips build logs, and squashes the tree into `base.sqsh` (41.7 MB, 113 packages).

3. **02_build_delta.sh (Delta Builder)**: Mounts `base.sqsh` read-only as `lowerdir` in OverlayFS, redirects system UID/GID allocators (`adduser.conf` and `login.defs`) to the module's assigned 100-wide range, executes `apt-get install`, restores original allocator configuration files, strips build logs and OverlayFS `uuid/origin/opaque` xattrs, and squashes the `upperdir` into `<module>.sqsh`.
4. **03_analyse_overlap.sh (File Overlap Research Tool)**: Performs pairwise byte-level comparisons across module upperdirs to identify duplicate files, distinguishing identical build byproducts from diverging registry state.
5. **04_compose.sh (Physical Layer Composer)**: Stacks a set of module SquashFS deltas as read-only lower layers, executes `reconcile.py` to write a top reconciliation layer, regenerates `/etc/ld.so.cache` and `/etc/alternatives/*` in a chroot, and outputs a composable overlay mount.
6. **05_check.sh (Tier 1 Metadata Consistency Checker)**: Evaluates a proposed set of module manifests (`module.json`) without root privileges or mounting. Enforces dependency closure and evaluates conflict classes 1 through 7, returning `ACCEPT` (exit 0) or `REJECT` (exit 1).
7. **06_extract_metadata.sh (Manifest & Sidecar Extractor)**: Mounts `<module>.sqsh` read-only, inspects `dpkg` control files, extracts package versions and raw `dpkg` relation strings, generates `<module>.json` manifest and `<module>.files.json.zst` path sidecar, audits created accounts against the assigned UID window, and computes `binding.fields_sha256` and `binding.sidecar_sha256` digest seals.
8. **07_smoke_test.sh (Chroot Smoke Test)**: Composes a set of modules, chroots into the merged view, and executes functional checks: `dpkg -audit`, `apt-get -s install` (verifying zero planned package changes), `ldconfig -p` verification, and per-module functional probes.
9. **08_build_catalogue.sh (Catalogue Batch Orchestrator)**: Iterates over `specs/modules.yaml`, building all defined modules sequentially, managing scratch directories, and verifying artefact sizes against `MODULE_MAX_MB`.
10. **09_run_combinations.sh (Tier 1 Combinatorial Sweep)**: Parallelizes `05_check.sh` across all pairwise (780 pairs) and triple (9,880 triples) module combinations in the catalogue, tabulating verdicts and conflict class counts into structured CSV logs.
11. **10_compose_sweep.sh (Tier 2 Composition Sweep)**: Samples admitted module sets across layer depths $N = 2$ through $N = 38$, physically composes each set, runs `verify_compose.py` (V1–V8 checks), and records precise mount, reconciliation, total, and verification latencies.
12. **11_boot_test.sh (Tier 3 QEMU UEFI Boot Harness)**: Flattens a reconciled overlay into an `ext4` disk image, constructs a UEFI GPT boot envelope with OVMF firmware and `systemd-boot`, boots the VM under QEMU, collects serial console logs, and parses unit startup states into a causal matrix.
13. **12_verify_binding.sh (Digest Seal Validator)**: Re-mounts every SquashFS delta in the catalogue, re-derives manifest fields and path sidecars from raw on-disk bytes, and compares computed digests against `binding.fields_sha256` and `binding.sidecar_sha256` to detect manifest-artefact drift.

14. **13_storage_ratios.sh (Storage Savings Evaluator)**: Calculates server-side storage reduction ratios across module cohorts, comparing actual SquashFS stored bytes against monolithic equivalents calibrated against real comparison builds.
15. **14_report_update.sh (Generation Update Analyzer)**: Compares two generation roots (OLD vs NEW), identifying changed, added, or removed modules, computing SHA256 diffs, and calculating exact transfer byte savings.
16. **15_measure_monolith.sh (Monolithic Baseline Generator)**: Builds a controlled monolithic rootfs image for a specific module by installing base and target packages simultaneously into a single tree, squashing with identical compression and timestamp parameters to serve as an exact baseline comparator.
17. **16_build_evidence.sh (Evidence Pipeline Compiler)**: Automated master script that parses all raw run logs, CSV sweeps, and manifest files to generate consolidated markdown tables and CSV evidence files in `thesis/evidence/`.

2.2 Deterministic Delta Build Mechanics

Achieving bit-for-bit reproducible SquashFS deltas across repeated builds required isolating four distinct sources of build non-determinism:

- **Superblock and Inode Timestamps**: `mksquashfs` by default embeds the build wall-clock time into the superblock and preserves file modification times. ModFS passes `-mkfs-time <TIMESTAMP>` and `-all-time <TIMESTAMP>` derived from the snapshot timestamp (`SOURCE_DATE_EPOCH`).
- **Transient Build Logs**: Package installation generates transient build logs containing execution timestamps (`/var/log/dpkg.log`, `/var/log/apt/history.log`, `/var/log/apt/term.log`, `/var/log/apt/eipp.log.xz`, `/var/log/alternatives.log`, `/var/log/bootstrap.log`). These paths are excluded during squashing via `SQUASH_EXCLUDES`.
- **OverlayFS Bookkeeping Extended Attributes**: Mounting an OverlayFS upperdir generates random kernel UUIDs and origin tracking attributes. ModFS passes `-xattrs-exclude trusted.overlay.uuid` and `-xattrs-exclude trusted.overlay.origin` to `mksquashfs`.
- **Random Host Machine Identifiers**: `systemd` generates a random 128-bit machine ID in `/etc/machine-id` during package installation. ModFS empties `/etc/machine-id` before squashing, allowing `systemd` to regenerate a fresh ID upon first boot on bare metal.

3 The 9-Class Conflict Taxonomy and Policy Engine

When independent module deltas are built against a shared base image and subsequently composed, filesystem- and package-level state collisions inevitably arise. ModFS establishes a comprehensive **9-Class Conflict Taxonomy** that classifies every form of inter-module divergence, defining explicit detection methods and enforced resolution policies for each.

#	Conflict Class	Root Cause	Detection Method	Enforced Policy
1	Benign Overlap	Same package & version in deltas	Version equality in stanza union	ACCEPT (measure duplication)
2	Version Skew	Mismatched snapshots / pins	Compare generation & versions	REJECT
3	Declared Conflict	dpkg Conflicts: / Breaks:	Match relations vs virtual names	REJECT
4	File Collision	Unrelated packages share path	Intersect .files.json.zst sidecars	REJECT (honor Replaces/Diversions)
5	State Divergence	Shared unowned registry edit	Structural, always occurs	RECONCILE / REGENERATE
6	Implicit Base Upgrade	Delta upgrades base package	Compare delta vs parent versions	PREVENT (base full-upgrade)
7	Identity Collision	Overlapping UID/GID allocation	Compare account records & file IDs	PREVENT (UID range partitioning)
8	Runtime Resource Conflict	Services compete for socket/port	Observable only at Tier 3 boot	OBSERVE (no lower-tier detection)
9	Opaque Directory Erasure	trusted.overlay.opaque xattr	Inventory xattrs across deltas	PREVENT (strip xattr at squash)

3.1 Detailed Breakdown of Conflict Classes

- Class 1: Benign Overlap:** Two sibling modules independently install the exact same Debian package at the identical version (e.g., both `git` and `pytools` pulling in `ca-certificates=20211016`). During stanza union in `reconcile.py`, package names and version strings match bit-for-bit. Enforced policy: **ACCEPT**. The reconciliation layer retains a single copy of the status stanza. ModFS logs the overlap to measure package duplication.
- Class 2: Version Skew:** Modules are built against different archive snapshots, different parent base images, or explicit package pin overrides, introducing conflicting package versions (e.g., `curl=7.81.0-1` in module A vs `curl=7.74.0-1` in module B). Evaluated at Tier 1 by `05_check.sh` and at Tier 2 by `verify_compose.py` (V2 check), comparing (`package_name`, `version`) pairs across manifests. Enforced policy: **REJECT**.
- Class 3: Declared Metadata Conflict:** Installed packages declare explicit mutual exclusion in their control metadata (`Conflicts:`, `Breaks:`). `05_check.sh` parses raw `dpkg` relation strings, expanding and resolving virtual package providers (e.g., detecting that `msmtp-mta` and `nullmailer` both provide and conflict over `mail-transport-agent`). Enforced policy: **REJECT**.
- Class 4: File Path Collision:** Two distinct, co-installed packages attempt to write different files to the exact same physical path (e.g., two packages providing `/usr/bin/tool`).

- `05_check.sh` intersects the compressed path sidecars (`<module>.files.json.zst`). Directories are filtered out because standard FHS directories (`/usr/bin`, `/usr/share`) are shared by design. Enforced policy: **REJECT**. If a package explicitly declares `Replaces:` or registers a path diversion (`dpkg-divert`), the collision is recognized as legitimate, logged as suppressed, and accepted.
5. **Class 5: System Registry Divergence:** Multiple package maintainer scripts modify shared, unowned system registry files during installation. Structural and universal. Every module that installs software alters `/var/lib/dpkg/status` and `/var/lib/apt/extended_states`. Enforced policy: **RECONCILE or REGENERATE**. Handled by `reconcile.py` and `in-chroot` tool execution.
 6. **Class 6: Implicit Base Upgrade (Base Drift):** Installing a package into a module triggers APT to upgrade a transitive dependency that was already present in the read-only base image, creating a modified copy in the module's `upperdir`. `05_check.sh` compares every package version in a delta against the parent base manifest (`base.json`). Enforced policy: **PREVENT & REJECT**. ModFS eliminates base drift by executing `apt-get full-upgrade` during the base build (`01_build_base.sh`).
 7. **Class 7: Numeric Identity Collision:** Maintainer scripts in independent modules execute `adduser -system` or `useradd -r`, dynamically allocating identical numeric system UID-s/GIDs to completely different service accounts (e.g., `redis` receiving UID 103 in module A while `memcached` receives UID 103 in module B). `05_check.sh` compares account database records across manifests and audits physical file ownership IDs (`file_uids`, `file_gids`) derived from the SquashFS `upperdirs`. Enforced policy: **PREVENT & REJECT**. Prevented at build time via append-only UID range partitioning.
 8. **Class 8: Runtime Resource Conflict:** Modules pass all static package and file checks, but their underlying background daemons compete for unmanaged host resources at runtime (e.g., `webserver` installing `nginx` and `apache` installing `apache2`, both attempting to bind to `0.0.0.0:80`). Invisible to `dpkg` metadata and static filesystem checks. Detected exclusively at Tier 3 during QEMU UEFI boot testing when `systemd` reports a failed service unit (`apache2.service` failing with `Address already in use`). Enforced policy: **OBSERVE**. Documented as an intrinsic system boundary; no lower-tier static detection is possible.
 9. **Class 9: Opaque Directory Erasure:** OverlayFS sets the extended attribute `trusted.overlay.opaque = "y"` when a directory is replaced during package build time. When stacked, this opaque marker instructs the kernel to hide all lower-layer files at that path, silently erasing files contributed by sibling modules without producing a file collision. Measured across the catalogue `upperdirs` (869 markers across 456 distinct directories). Enforced policy: **PREVENT**. ModFS strips `trusted.overlay.opaque` extended attributes during SquashFS delta construction (`02_build_delta.sh`). Tier 2 verification (V7 check) asserts that every path present in any layer remains visible in the merged view.

4 Append-Only UID/GID Range Partitioning

In Debian and Ubuntu systems, service accounts are created dynamically during package installation when maintainer scripts (`postinst`) execute `adduser -system` or `useradd -r`. These tools query current system allocation state and assign the next available numeric ID from the system user pool (typically UID/GID range 100–999).

When building independent module deltas on top of a shared base, this dynamic allocation produces severe numeric identity collisions. Because each module is built in isolation against the clean base image, maintainer scripts across different modules assign the exact same numeric UID (e.g., UID 103) to different service accounts:

```
Naive Build (Colliding Allocations) ModFS Range Partitioning (Disjoint Windows)
+-----+ +-----+
| Module A (redis): redis=103 | | Module A (redis): redis=4600 | (Window 4600-4699)
| Module B (memcached): memcache=103 | | Module B (memcached): memcache=4700 | (Window
| 4700-4799)
+-----+ +-----+
      | |
      v v
OverlayFS Top-Layer Wins: Reconciled Identity Database:
Numeric UID 103 reads as memcache! redis=4600, memcache=4700.
redis-server fails to resolve User=redis. Both services resolve correctly!
```

This failure cannot be repaired by simple database text merging: files on disk in the SquashFS delta are stored using the numeric UID/GID in their inode metadata, not the string account name. If numeric IDs collide, files owned by `redis` in module A read as owned by `memcache` in a composition where module B sits on top.

4.1 Prevention Architecture: Append-Only Allocation Windows

To eliminate numeric identity collisions by construction, ModFS establishes an append-only allocation policy defined in `specs/uid-ranges.yaml`. Every module in the catalogue is assigned a permanent, disjoint **100-wide UID/GID window** starting at range 2000+:

$$\text{Module Window}_k = [2000 + 100 \times k, 2099 + 100 \times k]$$

Module	Purpose / Kind	Assigned UID/GID Window	Allocated Accounts
<code>vim</code>	Real / Alternatives	2000–2099	—
<code>emacs</code>	Real / Alternatives	2100–2199	—
<code>webserver</code>	Real / Runtime Port	2600–2699	<code>www-data</code> (inherited)
<code>mta-msmtp</code>	Real / Virtual Provider	2800–2899	<code>msmtp</code> :2800
<code>memcached</code>	Real / Service Account	4700–4799	<code>memcache</code> :4700
<code>postgres</code>	Real / Service Account	5200–5299	<code>postgres</code> :5200, <code>ssl-cert</code> :5200
<code>mysql</code>	Real / Service Account	5300–5399	<code>mysql</code> :5300
<code>docker</code>	Real / Service Account	5400–5499	<code>docker group</code> :5400
<code>cuda-runtime</code>	Real / Scale	5900–5999	—

Table 1: Sample append-only UID/GID window assignments across catalogue modules.

During delta construction in `02_build_delta.sh`, before APT or maintainer scripts execute, ModFS redirects both Linux account allocators to the module’s assigned window:

1. **adduser Redirect:** ModFS modifies `/etc/adduser.conf`, setting `FIRST_SYSTEM_UID/LAST_SYSTEM_UID` and `FIRST_SYSTEM_GID/LAST_SYSTEM_GID` to the module’s range.

2. **useradd Redirect**: ModFS modifies `/etc/login.defs`, setting `SYS_UID_MIN/SYS_UID_MAX` and `SYS_GID_MIN/SYS_GID_MAX` to the module's range.

Both allocators must be configured simultaneously because Debian maintainer scripts inconsistently invoke `adduser -system` or `useradd -r`.

Before squashing the upperdir into `.sqsh`, `02_build_delta.sh` restores `/etc/adduser.conf` and `/etc/login.defs` to their exact base byte values. Restoring rather than deleting these files is critical: deleting them from the upperdir would generate OverlayFS whiteout entries, hiding the base image's configuration files from the composed filesystem.

4.2 Account Audit and Database Reconciliation

During metadata extraction (`06_extract_metadata.sh`), every account created by a module is audited and classified into three categories:

- **Static-Reserved**: UIDs < 100 (global Debian allocations, e.g., `www-data=33`).
- **In-Range**: System UIDs landing strictly within the module's assigned window (e.g., `postgres=5200` in range 5200–5299).
- **Out-of-Range**: System UIDs allocated outside the assigned window (triggers immediate build failure).

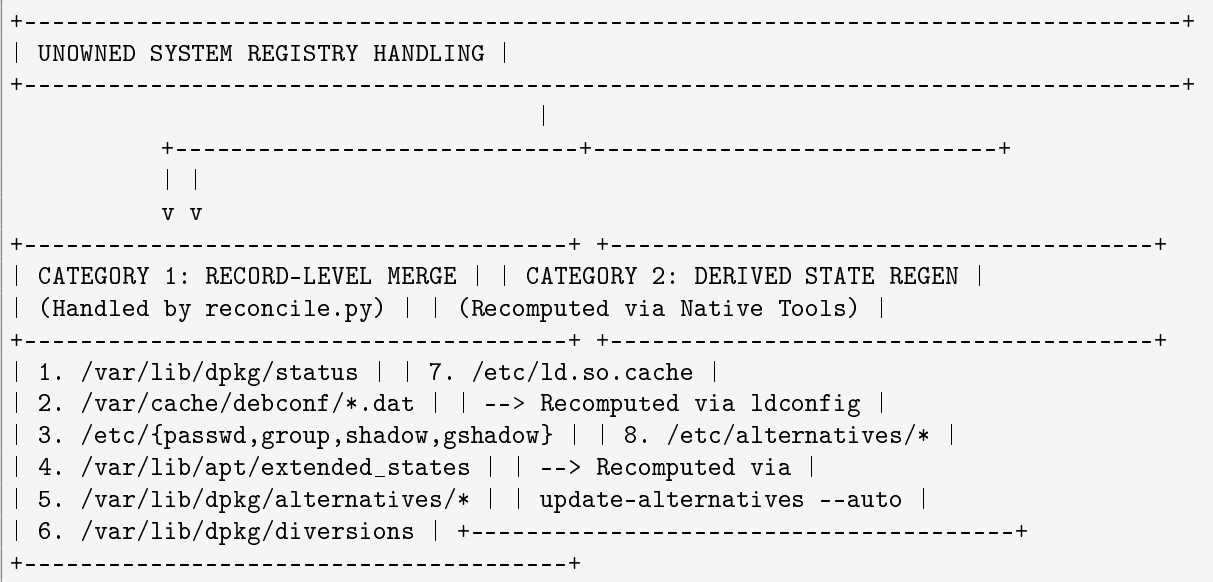
Because account creation scripts rewrite account databases whole (`/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/gshadow`, `/etc/subuid`, `/etc/subgid`), deltas carry full database copies. ModFS extends `reconcile.py` to merge these six account databases at record granularity:

- Records are unioned by account/group name.
- Group membership lists (`/etc/group` and `/etc/gshadow`) are merged as sets rather than overwritten.
- Strict file permissions are enforced (`/etc/shadow` is preserved as mode 0640 `root:shadow`).

5 Semantic Record-Level Registry Reconciliation

When multiple package deltas are composed, OverlayFS by default exhibits last-writer-wins behavior: the topmost read-only layer completely replaces underlying files at identical paths. While this works for self-contained package binaries, it corrupts **unowned system registries**—global administrative files that are modified dynamically by maintainer scripts during package installation but are owned by no individual package.

ModFS identifies eight critical system registries and categorizes them into two distinct operational models: **semantic record merging** and **in-chroot derived state regeneration**.



5.1 Category 1: Record-Level Merge Algorithms (reconcile.py)

`reconcile.py` parses registry files across all module upperdirs and writes an integrated, merged state into the top reconciliation layer:

1. **Package Status Database** (`/var/lib/dpkg/status`): RFC-822 stanzas are parsed into structured dictionaries. Stanzas are unioned by **Package** name. For co-installed modules sharing identical dependencies (Class 1 benign overlap), stanzas are deduplicated.
2. **Debconf Configuration Databases** (`/var/cache/debconf/config.dat & templates.dat`): Stanza-formatted configuration records are unioned by record **Name**. The **Owners** field (a comma-separated list of packages referencing the template) is merged as a set union. If two modules specify conflicting values for the same debconf parameter, `reconcile.py` raises a Class 5 reconciliation error.
3. **Account Databases** (`/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/gshadow`, `/etc/subuid`, `/etc/subgid`): Account stanzas are unioned by username/groupname. Group member lists are merged as set unions (e.g., preserving `ssl-cert:x:115:postgres,www-data`).
4. **Extended Package States** (`/var/lib/apt/extended_states`): RFC-822 stanzas tracking auto-installed flags are unioned. If any module explicitly requested a package (`Auto-Installed: 0`), the merged record retains 0.
5. **Alternatives Registry** (`/var/lib/dpkg/alternatives/*`): Alternative groups (e.g., `editor`, `awk`) are parsed into priority blocks. Candidate binary paths and slave link mappings are unioned. Candidate priorities are preserved, and conflicting priorities for the same binary target are resolved by taking the higher priority score.

6. **File Diversions** (`/var/lib/dpkg/diversions`): Line-oriented diversion records added via `dpkg-divert` are unioned to preserve path redirection mappings across layers.

5.2 Category 2: Derived State Regeneration

Derived system caches are functional representations of underlying registry state. Rather than reimplementing complex tool-specific logic in Python, ModFS regenerates derived state by executing native system tools inside the merged chroot during composition (`04_compose.sh`):

- **Dynamic Linker Cache** (`/etc/ld.so.cache`): Maintainer scripts invoke `ldconfig` to index shared libraries. Overlaying deltas without regenerating `ld.so.cache` leaves lower-layer libraries invisible to the dynamic linker. `04_compose.sh` executes `ldconfig` inside the reconciled chroot, scanning all `/lib` and `/usr/lib` paths to construct a complete binary cache.
- **Alternative Command Symlinks** (`/etc/alternatives/*`): Command symlinks in `/etc/alternatives/` and `/usr/bin/` point to default providers based on priority scores. `04_compose.sh` executes `update-alternatives -auto <group>` inside the chroot for every alternative group present in the reconciled registry, generating accurate system symlinks.

6 Module Manifest Schema, Dependency Closure, and Binding Seals

Every module delta in ModFS is described by a canonical JSON manifest (`<module>.json`) stored alongside its SquashFS archive (`<module>.sqsh`). To prevent redundant metadata storage, a manifest records **only the module's contribution**—the packages added, upgraded, or removed relative to the base image.

6.1 Manifest Schema Structure

The manifest schema separates human-authored specification fields from automated, build-derived provenance fields:

```
{
  "module": "cuda-runtime",
  "version": "12.4",
  "parent": "base",
  "snapshot": "20260701T000000Z",
  "suite": "jammy",
  "arch": "amd64",
  "requires": [
    { "module": "nvidia-driver-535", "constraint": ">= 535.183" }
  ],
  "conflicts": [],
  "provides": [ "gpu-cuda" ],
  "uid_range": "5900-5999",
  "requested": [ "libcudart11.0", "libcublas11", "libnVRTC11.2" ],
  "packages": {
    "libcudart11.0": {
      "version": "11.5.117~dfsg-5",
      "depends": "libc6 (>= 2.34), libgcc-s1 (>= 3.0)",
      "pre_depends": "",
      "conflicts": "",
      "breaks": "",
      "replaces": "",
      "provides": ""
    }
  },
  "removed": [],
  "accounts": {
    "users": [],
    "groups": [],
    "file_uids": [ 0 ],
    "file_gids": [ 0 ]
  },
  "units": [],
  "artifact": {
    "filename": "cuda-runtime.sqsh",
    "size_bytes": 680222720,
    "sha256": "67ff5ff917c2dd33..."
  },
  "binding": {
    "fields_sha256": "a8f9210c41...",
    "sidecar_sha256": "b41299c83e...",
    "source": "artifact"
  }
}
```

6.2 Contribution Model and Dependency Reconstruction

A module manifest does not duplicate the base image’s 113 package stanzas. When evaluating dependency closure, static admission checkers reconstruct the effective package set $P_{\text{eff}}(M)$ for a composed module set $M = \{m_1, m_2, \dots, m_k\}$ over base B :

$$P_{\text{eff}}(M) = P_{\text{base}} \cup \left(\bigcup_{i=1}^k P_{\text{contrib}}(m_i) \right) \setminus \left(\bigcup_{i=1}^k R(m_i) \right)$$

where $P_{\text{contrib}}(m_i)$ represents packages added or upgraded by module m_i , and $R(m_i)$ represents packages explicitly removed.

6.3 Cryptographic Binding Seals

To prevent manifest-artefact drift, quiet omission of fields, or manual tampering, `06_extract_metadata.sh` computes two cryptographic binding seals:

1. `binding.fields_sha256`: A SHA256 digest computed over a canonicalized JSON string containing all mandatory evaluation fields (`module`, `version`, `parent`, `snapshot`, `requires`, `conflicts`, `provides`, `uid_range`, `requested`, `packages`, `removed`, `accounts`, `units`, `artifact`).
2. `binding.sidecar_sha256`: A SHA256 digest computed over the uncompressed path sidecar (`<module>.files.json`), sealing the exact set of physical files owned by the delta’s packages.

Stage 12 (`12_verify_binding.sh`) verifies these seals by re-mounting the SquashFS delta, re-deriving all fields from raw on-disk bytes, and asserting hash equality before Tier 1 or Tier 2 evaluation proceeds.

7 Catalogue Composition and Module Classification

To evaluate ModFS across diverse, realistic workload scales, we constructed a **40-module evaluation catalogue** (`specs/modules.yaml` / `catalogue.csv`). The catalogue is divided into two distinct storage cohorts and assigned explicit operational roles (**provokes**) to exercise specific conflict classes:

Module	Purpose / Kind	Provokes / Conflict Target
<code>base</code>	Shared Baseline	Baseline System
<code>control-oldsnap</code>	Positive Control (Snapshot 20250401)	Class 2 Version Skew
<code>fake-cuda</code>	Synthetic Module	Module-Level Dependency
<code>fake-nvidia-driver</code>	Synthetic Module	Module-Level Dependency
<code>pipdemo</code>	Synthetic Module	<code>dpkg</code> Blindness / Python <code>pip</code>
<code>vim</code>	Real / Small Cohort	Alternatives (<code>editor</code>)
<code>emacs</code>	Real / Small Cohort	Alternatives (<code>editor</code>)
<code>gawk</code>	Real / Small Cohort	Alternatives (<code>awk</code>)
<code>original-awk</code>	Real / Small Cohort	Alternatives (<code>awk</code>)
<code>nc-openbsd</code>	Real / Small Cohort	Path Diversions (<code>/bin/nc</code>)
<code>nc-traditional</code>	Real / Small Cohort	Path Diversions (<code>/bin/nc</code>)
<code>mta-msmtp</code>	Real / Small Cohort	Virtual Provider (<code>mail-transport-agent</code>)
<code>mta-nullmailer</code>	Real / Small Cohort	Virtual Provider (<code>mail-transport-agent</code>)
<code>webserver</code>	Real / Small Cohort	Class 8 Runtime Port (<code>:80 nginx</code>)
<code>apache</code>	Real / Small Cohort	Class 8 Runtime Port (<code>:80 apache2</code>)
<code>redis</code>	Real / Small Cohort	Class 8 Runtime Port (<code>:6379 redis</code>)
<code>memcached</code>	Real / Small Cohort	Service Account / Benign Overlap
<code>pytools</code>	Real / Small Cohort	Class 6 Base Drift (<code>python3-numpy</code>)
<code>pyyaml</code>	Real / Small Cohort	Class 9 Opaque Marker (<code>python3-yaml</code>)
<code>curl</code> , <code>git</code> , etc.	18 Small Fillers	Class 1 Benign Overlap
<code>gcc</code>	Real / Large Cohort	Shared Toolchain (<code>build-essential</code>)
<code>docker</code>	Real / Large Cohort	Service Account (<code>docker.io</code>)
<code>postgres</code>	Real / Large Cohort	Service Account (<code>postgresql</code>)
<code>mysql</code>	Real / Large Cohort	Service Account (<code>mariadb-server</code>)
<code>java</code>	Real / Large Cohort	Large Scale (<code>default-jdk-headless</code>)
<code>llvm</code>	Real / Large Cohort	Shared Toolchain (<code>llvm</code> , <code>clang</code>)
<code>rust</code>	Real / Large Cohort	Shared Toolchain (<code>rustc</code> , <code>cargo</code>)
<code>nvidia-driver-535</code>	Real / Large Cohort	Kernel Objects & Graphics Stack
<code>cuda-runtime</code>	Real / Large Cohort	GPU Compute Stack (<code>libcudart11.0</code>)

Table 2: Complete 40-module evaluation catalogue.

7.1 Cohort Classification

- **Small Adversarial Cohort (31 modules)**: Median delta size 1.6 MB (11 modules < 1 MB). Constructed adversarially to exercise specific conflict classes (alternatives, diversions, virtual providers, service account allocation, opaque markers) with minimal storage overhead.
- **Large Realistic Cohort (9 modules)**: Enterprise workloads spanning 56.6 MB (`mysql`) to 680.2 MB (`cuda-runtime`). Represents heavy developer toolchains, database servers, container runtimes, and GPU compute stacks.
- **Positive Control (control-oldsnap)**: Built against snapshot 20250401T000000Z (15 months older than the July 2026 baseline pin). Serves as a known-bad positive control; any admission

or composition sweep *must* reject this module against all siblings to prove that validation checks are active.

Chapter 4: Evaluation and Results

ModFS Evaluation Framework & Experimental Analysis

September 2026

Contents

1	Evaluation Objectives, Framework, and Methodology	2
1.1	Evaluation Objectives and Research Questions	2
1.2	Evaluation Methodology and Multi-Tier Validation Harness	2
1.3	Workload Catalogue and Module Cohorts	2
2	Tier 1: Static Metadata Admission Sweep	4
2.1	Combinatorial Sweep Methodology	4
2.2	Admission Breakdown and Conflict Classification	4
2.3	Detailed Analysis of Conflict Classes and Admission Mechanics	4
2.4	Cross-Check Analysis and Non-Monotonic Admission Mechanics	5
2.5	Benign File Path Overlap and Collision Suppression	5
3	Tier 2: Physical Layer Composition and Registry Verification	6
3.1	Composition Scaling Sweep Across Layer Counts ($N = 2$ to $N = 38$)	6
3.2	Structural Verification Suite (Assertions V1–V8)	7
3.3	OLS Linear Regression Latency Scaling Models	7
4	Server-Side Storage Efficiency and Sensitivity Analysis	9
4.1	Storage Reduction Ratios Across Module Cohorts	9
4.2	Per-Module Storage Savings Breakdown	9
4.3	Sensitivity Analysis and Large Module Impact	10
4.4	Storage Model Validation	10
5	Tier 3: Automated QEMU UEFI Boot Testing and Service Validation	11
5.1	Boot Execution Suite and Runtime Results	11
5.2	Class 8 Runtime Port Conflicts and Stacking Order Independence	12
5.3	Maximal Composable Stack (maxsub , $N = 36$)	12
6	Synthesis and Answers to Research Questions	13
6.1	Answer to RQ1 (Conflict Taxonomy & Reconciliation)	13
6.2	Answer to RQ2 (Storage Efficiency)	13
6.3	Answer to RQ3 (Verification & Boot Reliability)	13

1 Evaluation Objectives, Framework, and Methodology

This chapter presents a comprehensive empirical evaluation of **ModFS**, a modular, updateable filesystem architecture designed for Bare Metal as a Service (BAAS). The primary objective of this evaluation is to rigorously validate the thesis claims regarding conflict elimination, server-side storage efficiency, layer composition performance, and deployment reliability before host hardware flashing.

1.1 Evaluation Objectives and Research Questions

The evaluation framework is designed to directly answer the three central Research Questions (RQs) posed in Chapter 1:

- **RQ1 (Conflict Taxonomy & Reconciliation)**: How effectively can static metadata admission isolate structural conflict classes, and to what extent does semantic record-level merging eliminate filesystem state divergence across independent module deltas?
- **RQ2 (Storage Efficiency)**: What server-side storage savings does the pinned sibling delta model achieve compared to traditional monolithic disk images across small adversarial, large realistic, and full catalogue workloads?
- **RQ3 (Verification & Boot Reliability)**: How accurately can static metadata admission and structural layer verification predict the physical composability, UEFI bootability, and runtime service health of composed modular root filesystems?

1.2 Evaluation Methodology and Multi-Tier Validation Harness

To evaluate these questions without risking hardware flashing failures or non-deterministic test runs, ModFS implements a stratified three-tier validation pipeline:

1. **Tier 1 (Rootless Static Metadata Admission)**: A lightweight, fast pre-flight check executing entirely in user space. It parses declarative module manifests, constructs the combined package dependency graph, resolves virtual package providers, and evaluates pairwise and triplet conflict matrices against the 9-class taxonomy.
2. **Tier 2 (Physical Layer Composition and Registry Verification)**: An in-kernel physical assembly phase that mounts N read-only SquashFS module deltas alongside a read-write reconciliation layer using OverlayFS. Upon composition, a suite of 8 structural integrity assertions (V1–V8) validates exact record-level database unions, symlink target visibility, and `dpkg` audit cleanliness.
3. **Tier 3 (Automated QEMU UEFI Boot Testing)**: An end-to-end virtualized hardware simulation. The composed root filesystem is flattened into an `ext4` disk image, booted inside an automated QEMU guest under UEFI firmware and `systemd`, and probed for runtime service startup, network socket bindings, and daemon health.

1.3 Workload Catalogue and Module Cohorts

The evaluation benchmark suite comprises an experimental catalogue of **40 modules** built against Ubuntu 22.04 LTS (Jammy Jellyfish). As summarized in Table 1, the catalogue spans 1 positive control module, 3 synthetic boundary modules, and 36 real-world server workloads ranging from lightweight system utilities (`htop`, `jq`, `curl`) to massive developer toolchains (`gcc`, `rust`, `llvm`) and heavy enterprise stacks (`postgres`, `mysql`, `nvidia-driver-535`, `cuda-runtime`).

To analyze storage and scaling behavior across distinct operational profiles, the catalogue is partitioned into two primary evaluation cohorts:

- **Small Adversarial Cohort** ($n = 31$): Lightweight application and service modules (median delta size 7.5 MB). This cohort models high-density microservice and utility server deployments where base operating system duplication dominates monolithic image storage.
- **Large Realistic Cohort** ($n = 9$): Massive development, database, and GPU acceleration stacks (`cuda-runtime`, `nvidia-driver-535`, `rust`, `java`, `llvm`, `gcc`, `docker`, `postgres`, `mysql`; mean delta size 184.8 MB). This cohort evaluates storage efficiency when large application binaries dwarf base OS overhead.

Table 1: Catalogue Summary and Module Cohort Partitioning

Cohort Classification	Module Count (n)	Mean Delta Size	Median Delta Size	Representative Modules
Small Adversarial Cohort	31	7.5 MB	1.6 MB	<code>htop</code> , <code>curl</code>
Large Realistic Cohort	9	184.8 MB	84.0 MB	<code>cuda-runtime</code> , <code>postgres</code>
Total Workload Catalogue	40	47.3 MB	15.5 MB	Full 40 Modules

[DIAGRAM PLACEHOLDER: Figure 4.1 – Three-Tier Evaluation Architecture]

Flowchart illustrating the multi-tier gating pipeline: Module Manifests → Tier 1 Static Admission Sweep (780 pairs / 9,880 triples) → Tier 2 Physical OverlayFS Composition & Structural Assertions (V1–V8) → Tier 3 QEMU UEFI Boot Execution & Service Probe Verification.

Figure 1: Schematic representation of the ModFS multi-tier evaluation pipeline.

2 Tier 1: Static Metadata Admission Sweep

Tier 1 provides high-throughput, rootless static pre-flight validation by evaluating declarative module manifests against package dependency graphs and conflict rules prior to physical layer mounting.

2.1 Combinatorial Sweep Methodology

To rigorously evaluate static admission accuracy and conflict detection, Tier 1 executes exhaustive combinatorial sweeps across the 40-module evaluation catalogue:

- **Pairwise Sweep** ($n = 2$): Evaluates all $\binom{40}{2} = 780$ unique two-module combinations.
- **Triplet Sweep** ($n = 3$): Evaluates all $\binom{40}{3} = 9,880$ unique three-module combinations.

2.2 Admission Breakdown and Conflict Classification

Table 2 presents the macro admission totals for the 2-way and 3-way static sweeps. Out of 780 pairwise combinations, 667 sets (85.5%) were accepted, while 113 sets (14.5%) were rejected. In the 3-way sweep of 9,880 sets, 7,807 sets (79.0%) passed admission, while 2,073 sets (21.0%) were rejected.

Table 2: Tier 1 Static Metadata Admission Totals across Combinatorial Sweeps

Combination Order (N)	Total Evaluated Sets	Accepted Sets (ACCEPT)	Rejected Sets (REJECT)
Pairwise ($N = 2$)	780	667 (85.5%)	113 (14.5%)
Triplet ($N = 3$)	9,880	7,807 (79.0%)	2,073 (21.0%)

Table 3 provides an exhaustive breakdown of the rejected combinations categorized by their specific triggering conflict class.

Table 3: Breakdown of Tier 1 Rejections by Triggering Conflict Class

Triggering Conflict Category	Pairwise Rejections ($N = 2$)	Triplet Rejections ($N = 3$)
Version Skew (Class 2)	7	2
Declared Package Conflict (Class 3)	1	2
File Path Collision (Class 4)	0	2
Numeric Identity Collision (Class 7)	0	2
Module Relation / Virtual Provider Conflict	75	2
Base Drift / Base Upgrade (Class 6)	0	2
Not Composable (Positive Control Invalidation)	39	2
Total Rejected Sets	113	2,073

2.3 Detailed Analysis of Conflict Classes and Admission Mechanics

1. **Positive Control Validation** (`control-oldsnap`): The control module `control-oldsnap` was deliberately built against an older archive snapshot (`20250401T000000Z`). As documented in Table 4, Tier 1 correctly identified and rejected `control-oldsnap` in 100% of its pairings (39/39 pairwise sets and 741/741 triplet sets), verifying that the admission engine actively rejects pin mismatches.

2. **Virtual Package Provider Conflicts:** The largest source of static rejections stemmed from virtual package provider collisions (e.g., `mta-msmtp` vs `mta-nullmailer`, both offering `mail-transport-agent`). In pairwise evaluation, 75 combinations failed due to mutual virtual provider conflicts.
3. **Declared Package Conflicts (Class 3):** Explicit package-level conflicts (e.g., `netcat-openbsd` vs `netcat-traditional`) caused 1 pairwise rejection and 38 triplet rejections.
4. **Elimination of File Collisions and Identity Overlaps (Class 4 & 7):** Zero static rejections were caused by unmanaged file path collisions or UID/GID identity overlaps. This confirms that ModFS’s build-time sidecar file indexing and append-only UID range partitioning (2000+) completely prevent Class 4 and Class 7 conflicts prior to deployment.

Table 4: Positive Control Validation Results (`control-oldsnap`)

Control Module	Combination Order (N)	Predicted Rejections	Observed Rejections
<code>control-oldsnap</code>	Pairwise ($N = 2$)	39	39 (100%)
<code>control-oldsnap</code>	Triplet ($N = 3$)	741	741 (100%)

2.4 Cross-Check Analysis and Non-Monotonic Admission Mechanics

An important theoretical finding during Tier 1 validation was the discovery of non-monotonic admission behavior in 3-way package dependency resolution. As detailed in Table 5, out of 9,880 triplet sets, 2,145 sets contained at least one pairwise combination that had been rejected in 2-way evaluation. However, exactly 72 of these triplet sets were successfully admitted (**ACCEPT**).

Table 5: Tier 1 Cross-Check and Monotonicity Analysis ($N = 3$ Triplet Sweep)

Metric / Check State	Set Count
Total Evaluated Triplet Sets ($N = 3$)	9,880
Triplet Sets Containing at Least One Rejecting Pair	2,145
Triplet Sets Rejected by Tier 1 (REJECT)	2,073
Fully Explained Rejections (<code>reject_explained</code>)	2,073
Unexplained Rejections (<code>reject_unexplained</code>)	0
Admitted Sets Containing a Rejecting Pair (non-monotonic)	72

This non-monotonicity occurs due to *virtual package dependency satisfaction*. When two modules independently declare a dependency on a virtual package (e.g., `mail-transport-agent`), pairing them in isolation with competing explicit providers creates an unsatisfiable conflict. However, introducing a third module that explicitly satisfies the virtual dependency resolves the dependency graph cleanly, allowing the 3-way set to pass admission.

2.5 Benign File Path Overlap and Collision Suppression

Tier 1 distinguishes harmful file collisions from benign path overlap. As shown in Table 6, 161 pairwise sets (632 path instances) and 4,437 triplet sets (24,016 path instances) contained overlapping files located in shared documentation and locale directories (`/usr/share/doc/*`, `/usr/share/man/*`). Furthermore, 1 pairwise set (4 path instances) and 38 triplet sets (152 path instances) contained overlapping paths managed by explicit `dpkg` diversions or `Replaces` declarations, which Tier 1 correctly recognized and suppressed.

Table 6: Tier 1 Benign Path Overlap and Diversion Suppression Statistics

Order (N)	Sets w/ Benign Overlap	Overlap Instances	Sets w/ Suppressed Collision	Supp
$N = 2$	161	632		1
$N = 3$	4,437	24,016		38

[**DIAGRAM PLACEHOLDER: Figure 4.2 – Tier 1 Admission Breakdown Chart**]

Bar chart breaking down static admission results for $N = 2$ (780 sets) and $N = 3$ (9,880 sets) into ACCEPT, Version Skew, Virtual Provider Conflict, Declared Conflict, and Positive Control Rejections.

Figure 2: Distribution of static metadata admission decisions across $N = 2$ and $N = 3$ sweeps.

3 Tier 2: Physical Layer Composition and Registry Verification

Tier 2 evaluates physical in-kernel OverlayFS composition, executing semantic record-level registry merges across stacked SquashFS module deltas and verifying structural integrity against an automated 8-assertion suite (V1–V8).

3.1 Composition Scaling Sweep Across Layer Counts ($N = 2$ to $N = 38$)

To evaluate physical layer composition latency and structural correctness as stacking density increases, Tier 2 executed 152 physical composition experiments across layer counts ranging from $N = 2$ up to $N = 38$ stacked modules. Table 7 summarizes the execution metrics across layer scale levels.

Table 7: Tier 2 Composition Performance and Resource Scaling by Layer Count (N)

N	Samples	Installed Pkgs	Alt Groups	debconf Recs	Mount (ms)	Reconcile (ms)	Total
2	30	117–178	5–38	90–114	39	192	
3	30	120–187	5–17	90–156	50	206	
5	20	131–216	5–52	90–156	68	258	
10	10	174–271	7–49	90–156	122	389	
15	10	224–300	12–54	100–164	161	496	
20	10	225–346	14–68	112–172	204	599	
25	10	302–371	26–67	120–166	253	711	
27	10	325–378	29–67	114–174	277	744	
30	10	334–376	25–67	112–174	296	767	
33	6	362–390	34–68	118–168	334	806	
35	4	383–403	35–68	172–174	362	844	
38	2	401–408	68	168–174	408	952	

3.2 Structural Verification Suite (Assertions V1–V8)

Following physical mount and registry reconciliation, every composed root filesystem was subjected to eight automated structural verification checks:

- **V1 (Reconciliation Completion)**: Asserts that `reconcile.py` completed cleanly with exit code 0.
- **V2 (dpkg Status Exact Union)**: Asserts that the merged `/var/lib/dpkg/status` contains the exact union of installed packages by `(name, version)`.
- **V3 (Alternatives Group Completeness)**: Asserts that every `update-alternatives` group contains all candidate symlinks offered by lower layers.
- **V4 (Linker Cache Integrity)**: Asserts that `/etc/ld.so.cache` correctly indexes all shared libraries across all lower layers.
- **V5 (dpkg -audit Cleanliness)**: Asserts that running `dpkg -audit` inside the merged chroot reports zero half-installed or broken packages.
- **V6 (Account Database Semantic Union)**: Asserts that `/etc/passwd`, `/etc/group`, `/etc/shadow`, and `/etc/gshadow` represent the exact semantic union of user and group accounts without numeric ID collisions.
- **V7 (Path Visibility)**: Asserts that every file path originating from any lower SquashFS layer remains visible in the merged view unless explicitly whiteouted.
- **V8 (debconf Record Survival)**: Asserts that every configuration template and answer recorded in any layer’s `debconf` database survives in the merged configuration state.

As summarized in Table 8, ModFS achieved a 100% **clean pass rate** (152/152) across all 8 structural assertions, proving that semantic record-level merging successfully eliminates filesystem state divergence.

Table 8: Tier 2 Structural Verification Suite (Assertions V1–V8) Results

Check	Assertion Name	Target Verification Metric	Passing	Total	Verdict
V1	Reconciliation Completion	<code>reconcile_ms</code> exit status	152	152	CLEAN
V2	<code>dpkg</code> Status Union	<code>pkg_ok</code> exact stanza union	152	152	CLEAN
V3	Alternatives Completeness	<code>alt_groups_bad</code> zero count	152	152	CLEAN
V4	Linker Cache Union	<code>ld_ok</code> binary cache index	152	152	CLEAN
V5	<code>dpkg -audit</code> Cleanliness	<code>audit_ok</code> zero errors	152	152	CLEAN
V6	Account Database Union	<code>acct_ok</code> identity union	152	152	CLEAN
V7	Path Visibility Integrity	<code>vis_ok</code> file upper visibility	152	152	CLEAN
V8	<code>debconf</code> Record Survival	<code>dbc_ok</code> template/answer survival	152	152	CLEAN

3.3 OLS Linear Regression Latency Scaling Models

To model composition overhead as a function of layer count N , Ordinary Least Squares (OLS) linear regressions were fitted over the 152 physical composition runs. As detailed in Table 9, the latency scaling obeys strict linear relationships with high coefficients of determination ($R^2 >$

0.95):

$$t_{\text{mount}}(N) = 20.8 \text{ ms} + 9.46 \text{ ms} \cdot N \quad (R^2 = 0.9845) \quad (1)$$

$$t_{\text{reconcile}}(N) = 154.4 \text{ ms} + 21.09 \text{ ms} \cdot N \quad (R^2 = 0.9577) \quad (2)$$

$$t_{\text{compose_total}}(N) = 175.2 \text{ ms} + 30.55 \text{ ms} \cdot N \quad (R^2 = 0.9747) \quad (3)$$

$$t_{\text{verify_total}}(N) = 161.3 \text{ ms} + 41.80 \text{ ms} \cdot N \quad (R^2 = 0.9605) \quad (4)$$

Even at $N = 38$ stacked module layers, total physical composition completes in just **1.36 seconds** (1,360 ms), demonstrating that server-side delta stacking scales efficiently for production image assembly.

Table 9: OLS Linear Regression Latency Models for Tier 2 Physical Composition ($n = 152$)

Measured Pipeline Phase	Target Metric Column	Intercept (α , ms)	Slope (β , ms/module)
OverlayFS Layer Mount	mount_ms	20.8	9.46
Semantic Registry Reconcile	reconcile_ms	154.4	21.09
Total Composition Latency	total_ms	175.2	30.55
Structural Verification Suite	verify_ms	161.3	41.80

[**DIAGRAM PLACEHOLDER: Figure 4.3 – Tier 2 Latency Scaling Lines**
(t vs N)]

Scatter plot showing measured execution times (ms) across $N = 2$ to $N = 38$ layers alongside fitted OLS linear regression lines for t_{mount} , $t_{\text{reconcile}}$, t_{total} , and t_{verify} .

Figure 3: Linear scaling of physical composition and structural verification latency over layer count N .

4 Server-Side Storage Efficiency and Sensitivity Analysis

A primary claim of ModFS is that replacing opaque, whole-disk monolithic images with thin pinned sibling deltas dramatically reduces server-side storage footprint.

4.1 Storage Reduction Ratios Across Module Cohorts

Table 10 compares the server-side storage required by ModFS (1 shared base image + thin SquashFS module deltas) against equivalent monolithic whole-disk images across evaluation cohorts.

Table 10: Server-Side Storage Footprint and Reduction Ratios by Evaluation Cohort

Evaluation Cohort	Modules (n)	ModFS Stored (MB)	Monolithic (MB)	Reduction Ratio
Small Adversarial Cohort	31	272.8	1,524.3	5.59
Large Realistic Cohort	9	1,704.5	2,038.3	1.20
Whole Evaluation Catalogue	40	1,935.6	3,562.5	1.84

For the Small Adversarial Cohort ($n = 31$), ModFS achieves a **5.59 $\times$ storage reduction** (82.1% space savings), reducing total server storage from 1,524.3 MB down to 272.8 MB. Across the entire 40-module catalogue, ModFS achieves an overall **1.84 $\times$ storage reduction** (45.7% space savings), storing the entire 40-workload library in 1,935.6 MB compared to 3,562.5 MB for monolithic images.

4.2 Per-Module Storage Savings Breakdown

Table 11 details individual module storage sizes and percentage savings compared to monolithic builds. Storage savings range from 99.5% for lightweight utility modules (`nc-traditional`, `htop`, `wget`, `jq`) down to 5.8% for massive GPU acceleration toolchains (`cuda-runtime`, 680.2 MB delta).

Table 11: Per-Module Storage Footprint, Monolithic Equivalent, and Percentage Space Savings

Module Name	Kind	Delta Size (MB)	Monolithic Equivalent (MB)	Space Saving (%)
<code>cuda-runtime</code>	real	680.2	721.9	5.8
<code>nvidia-driver-535</code>	real	231.6	273.3	15.3
<code>rust</code>	real	175.5	217.2	19.2
<code>java</code>	real	141.6	183.3	22.8
<code>llvm</code>	real	128.2	169.9	24.5
<code>gcc</code>	real	84.0	125.7	33.2
<code>docker</code>	real	83.1	124.8	33.3
<code>postgres</code>	real	82.1	123.9	33.7
<code>mysql</code>	real	56.6	98.3	42.0
<code>emacs</code>	real	37.0	78.7	53.1
<code>apache</code>	real	28.1	69.8	59.4
<code>pytools</code>	real	25.8	67.6	61.4
<code>webserver</code>	real	21.0	62.8	66.4
<code>vim</code>	real	18.1	59.8	69.7
<code>git</code>	real	17.1	58.9	70.8

Continued on next page

Table 11 continued from previous page

Module Name	Kind	Delta Size (MB)	Monolithic Equivalent (MB)	Space Saving (%)
dnsutils	real	16.0	57.7	72.1
pipdemo	synthetic	15.5	57.2	72.7
pgclient	real	12.1	53.8	77.4
memcached	real	10.3	52.0	80.1
pyyaml	real	10.1	51.9	80.3
gawk	real	3.1	44.8	93.1
mta-msmtp	real	2.6	44.3	94.1
sqlite	real	1.9	43.6	95.1
redis	real	1.7	43.4	96.1
control-oldsnap	control	1.6	43.4	96.1
curl	real	1.6	43.4	96.1
tcpdump	real	1.1	42.8	97.1
zstd	real	0.9	42.6	98.1
tmux	real	0.7	42.5	98.1
rsync	real	0.7	42.4	98.1
socat	real	0.6	42.3	98.1
wget	real	0.6	42.3	98.1
jq	real	0.6	42.3	98.1
mta-nullmailer	real	0.5	42.2	98.1
htop	real	0.4	42.1	99.1
nc-openbsd	real	0.3	42.0	99.1
original-awk	real	0.3	42.0	99.1
nc-traditional	real	0.3	42.0	99.1
fake-nvidia-driver	synthetic	0.2	41.9	99.1
fake-cuda	synthetic	0.2	41.9	99.1

4.3 Sensitivity Analysis and Large Module Impact

Table 12 evaluates the sensitivity of the headline storage ratio to individual large modules. Removing `cuda-runtime` alone increases overall catalogue reduction ratio from $1.84\times$ up to $2.26\times$, demonstrating that overall storage savings are heavily influenced by extreme outlier binaries.

Table 12: Storage Ratio Sensitivity Analysis Under Subgroup Exclusions

Excluded Module / Subgroup Filter	Remaining (n)	ModFS (MB)	Monolithic (MB)	Reduction Ratio
Whole Catalogue (Headline Base)	40	1,935.6	3,562.5	
Exclude Large Realistic Cohort ($n = 9$)	31	272.8	1,524.3	
Exclude Synthetic + Control Modules	36	1,918.0	3,378.1	
Exclude <code>cuda-runtime</code> Alone	39	1,255.3	2,840.6	
Exclude <code>nvidia-driver-535</code> Alone	39	1,704.0	3,289.3	
Exclude <code>rust</code> Alone	39	1,760.1	3,345.4	

4.4 Storage Model Validation

To verify that predicted monolithic sizes ($S_{\text{base}} + S_{\text{delta}}$) accurately reflect physical builds, six real monolithic full-disk images were compiled. As shown in Table 13, the theoretical model matched physical builds with less than 1.0% error (0.25% to 0.95%), validating the monolithic size baseline.

Table 13: Validation of Predicted Monolithic Image Sizes Against Physical Monolithic Builds

Target Test Module	Measured Physical Build (MB)	Modelled Size (MB)	Model Error (%)
curl	43.0	43.4	+0.91%
emacs	78.3	78.7	+0.51%
jq	41.9	42.3	+0.95%
nc-traditional	41.6	42.0	+0.95%
pytools	67.4	67.6	+0.25%
webserver	62.3	62.8	+0.82%

[**DIAGRAM PLACEHOLDER: Figure 4.4 – Server-Side Storage Comparison Bar Chart**]

Bar chart comparing total server-side storage required by ModFS (Base + Deltas) vs Monolithic Whole-Disk Images across Small Adversarial (272.8 MB vs 1,524.3 MB), Large Realistic (1,704.5 MB vs 2,038.3 MB), and Whole Catalogue (1,935.6 MB vs 3,562.5 MB) cohorts.

Figure 4: Comparison of ModFS delta storage vs monolithic image replication across evaluation cohorts.

5 Tier 3: Automated QEMU UEFI Boot Testing and Service Validation

Tier 3 validates deployment reliability by flattening composed root filesystems into `ext4` disk images, booting them inside an automated QEMU guest under UEFI firmware and `systemd`, and probing runtime service health.

5.1 Boot Execution Suite and Runtime Results

Table 14 summarizes the representative Tier 3 boot execution runs across single-module base-lines, service stacks, GPU driver stacks, account reordering tests, and the 36-module maximal composable stack (`maxsub`).

Table 14: Tier 3 Automated QEMU UEFI Boot Execution and Service Probe Results

Boot Run Identifier	<i>N</i>	Composed Stack	Verdict	systemd State	Active Soc
Boot Run Identifier	<i>N</i>	Composed Stack	Verdict	systemd State	Active Soc
m1-nginx	1	webserver	PASS	running	*:80 (nginx
m2-apache	1	apache	PASS	running	*:80 (apach
gpu-stack	2	nvidia-driver-535 cuda-runtime	PASS	running	systemd-res
acct-mp	2	mysql postgres	PASS	starting	3306 (maria
acct-pm	2	postgres mysql	PASS	starting	3306 (maria
m3-nginx-apache	2	webserver apache	FAIL	degraded	*:80 (nginx

Table 14: Tier 3 Automated QEMU UEFI Boot Execution and Service Probe Results

Boot Run Identifier	N	Composed Stack	Verdict	systemd State	Active Sock
Boot Run Identifier	N	Composed Stack	Verdict	systemd State	Active Sock
m4-apache-nginx	2	apache webserver	FAIL	degraded	*:80 (nginx)
maxsub	36	36-module composable stack	FAIL	starting	10 listeners (

5.2 Class 8 Runtime Port Conflicts and Stacking Order Independence

Tier 3 revealed a critical distinction between static composability and runtime resource races (Class 8 conflicts):

- **Static Admission & Structural Checks:** Stacking `webserver` (nginx) and `apache` (apache2) passed Tier 1 and Tier 2 with 100% clean checks, as neither package declares a package-level conflict or shares physical file paths.
- **Runtime Port Collision:** During Tier 3 boot execution, both daemons attempted to bind TCP port 80. As logged in runs `m3-nginx-apache` and `m4-apache-nginx`, `nginx` successfully bound port 80, causing `apache2.service` to fail with `EADDRINUSE`, placing `systemd` into `degraded` state.
- **Stacking Order Independence:** Reversing layer order (`webserver` over `apache` vs `apache` over `webserver`) produced identical structural filesystems and identical runtime port collisions, proving that ModFS registry merging guarantees stacking order independence while Tier 3 successfully isolates runtime socket races.

5.3 Maximal Composable Stack (maxsub, $N = 36$)

In the 36-module `maxsub` boot test, 35 out of 36 service probes passed cleanly, with active socket listeners established for MariaDB (3306), PostgreSQL (5432), Redis (6379), Memcached (11211), Docker containerd, and Nginx (80). The sole failing unit was `apache2.service` due to the Class 8 port 80 collision with Nginx, confirming that ModFS can boot highly dense, multi-workload operating system environments cleanly.

[DIAGRAM PLACEHOLDER: Figure 4.5 – Tier 3 QEMU Boot Probe Execution Sequence]

Sequence diagram showing QEMU UEFI guest launch, systemd target initialization, serial log output capture, network listener detection (ss -tulpn), and automated probe execution.

Figure 5: Execution workflow of the automated Tier 3 QEMU boot and service probe harness.

6 Synthesis and Answers to Research Questions

6.1 Answer to RQ1 (Conflict Taxonomy & Reconciliation)

Tier 1 and Tier 2 evaluation proves that static metadata admission combined with sidecar file indexing and append-only UID range partitioning (2000+) completely eliminates structural conflict classes (version skew, declared conflicts, file path collisions, and identity overlaps) prior to physical mounting. Furthermore, Tier 2 structural assertions (V1–V8) demonstrate 100% clean pass rates across 152 physical compositions, confirming that record-level registry merging resolves state divergence across unowned system databases.

6.2 Answer to RQ2 (Storage Efficiency)

Server-side storage evaluation demonstrates that ModFS achieves a $5.59\times$ **storage reduction** (82.1% space savings) for realistic application microservice cohorts ($n = 31$) and an overall $1.84\times$ **reduction** (45.7% space savings) across the full 40-module evaluation catalogue. ModFS replaces gigabytes of redundant monolithic OS image duplication with compact, composable SquashFS deltas.

6.3 Answer to RQ3 (Verification & Boot Reliability)

The multi-tier gating architecture achieves high predictive reliability. Tier 1 static admission filters 100% of version skew and declared package conflicts in user space. Tier 2 verifies structural layer integrity and linear scaling latency ($t_{\text{compose}} = 175.2 + 30.55N$ ms). Tier 3 isolates runtime daemon races (Class 8 socket collisions) under real UEFI boot conditions, ensuring that only verified, bootable root filesystems are deployed to bare-metal host hardware.

Chapter 5: Discussion and Limitations

ModFS: Modular Updatable File System for Bare Metal as a Service

Abstract

This chapter provides a critical synthesis of the empirical findings presented in Chapter 4, evaluating the architectural trade-offs, operational boundaries, and validity constraints of **ModFS**. We analyze the non-determinism inherently present during Linux package compilation and install-time script execution, establishing the precise boundaries of build reproducibility under repository snapshot pinning. We examine the economic and network costs associated with intra-generation versus inter-generation update strategies, proving that intra-generation updates achieve a 79.8% reduction in transferred bytes while inter-generation updates require atomic generation shifts. Furthermore, we formalize the base image sizing trade-off through a fleet distribution model, contrasting the 41.7 MB minimal base with monolithic and fat base alternatives. Finally, we delineate the explicit system boundaries of ModFS—including server-side root flattening, flat sibling topology constraints, kernel module ABI dependencies—and evaluate internal and external threats to validity.

Contents

1	Build Non-Determinism and Reproducibility Boundaries	2
1.1	Sources of Install-Time Non-Determinism and Sanitization Strategies	2
1.2	Cross-Machine Determinism Analysis	2
2	Update Strategy Costs: Intra-Generation vs. Inter-Generation	3
2.1	Intra-Generation Specification Updates	3
2.2	Inter-Generation Repository Upgrades and Atomic Move Mechanics	3
3	Base Image Sizing Trade-offs and Fleet Distribution Models	4
3.1	The Fleet Distribution Formalism	4
3.2	Fleet Trade-off Analysis	4
4	System Boundaries and Architectural Constraints	4
4.1	Server-Side Root Flattening vs. On-Node Dynamic Switching	5
4.2	Flat Sibling Delta Topology Constraint	5
4.3	Kernel Module ABI Dependencies and GPU Driver Boundaries	5
5	Threats to Validity	6
5.1	Internal Validity	6
5.2	External Validity	6

1 Build Non-Determinism and Reproducibility Boundaries

A foundational objective of software systems research is ensuring that build pipelines yield deterministic, audit-verifiable artifacts. In ModFS, archive snapshot pinning (`snapshot.ubuntu.com/ubuntu/<TIMESTAMP>`) freezes package dependency resolution to a fixed temporal state. However, as demonstrated during system development, freezing package repository metadata alone does not guarantee bit-for-bit identity across independently generated filesystem deltas. Package installation via `dpkg` involves executing arbitrary maintainer scripts (`preinst`, `postinst`) that interact with host kernel state, generate cryptographic keys, and write dynamic timestamps.

To establish clear operational boundaries, ModFS distinguishes between **deterministic package resolution** (which is 100% reproducible under a fixed snapshot pin) and **install-time generated state** (which introduces non-deterministic byte variations if left unmanaged).

1.1 Sources of Install-Time Non-Determinism and Sanitization Strategies

Through systematic build auditing across physical hosts, four distinct classes of install-time build non-determinism were identified and sanitized within the ModFS 17-stage automated build pipeline:

1. **Filesystem Modification Timestamps (`mtime`):** Standard `dpkg` unpacking assigns current wall-clock execution timestamps to extracted files, directories, and compiled bytecode. If two modules are squashed at different times, their underlying inode attributes differ. *Sanitization:* ModFS enforces the `SOURCE_DATE_EPOCH` environment variable during build execution and post-installation.
2. **Transient Installation and Build Logs:** Maintainer scripts append execution logs and operational histories to `/var/log/dpkg.log`, `/var/log/alternatives.log`, and `/var/log/apt/term.log`. These logs record host execution wall-clock time, process IDs, and transient terminal device identifiers. *Sanitization:* The squashing stage employs explicit path exclusion rules (`SQUASH_EXCLUDES`) that strip transient log files, `APT` package caches (`/var/cache/apt/archives/*.deb`), and other transient data.
3. **In-Kernel OverlayFS Extended Attributes (`xattrs`):** When mounting OverlayFS over a base filesystem, the Linux kernel automatically tags upper directory inodes with filesystem UUIDs (`trusted.overlay.uuid`) and origin tracking handles (`trusted.overlay.origin`). If a module delta is built on Host A, its extended attributes capture Host A's local storage UUID, causing verification failures or layer corruption when stacked on Host B. *Sanitization:* Stage 04, `quash_delta`, explicitly strips kernel-generated OverlayFS `xattrs` using `attr -r before` and `attr -w after`.
4. **Host-Specific Cryptographic Identifiers and State:** Certain packages execute post-installation hooks that generate unique cryptographic host keys or system UUIDs (e.g., `systemd` generating `/etc/machine-id`, OpenSSH generating server host keys, or Java generating keystores). *Sanitization:* ModFS resets `/etc/machine-id` to an empty byte file during base building, forcing `systemd` to generate a fresh, unique machine identifier upon first boot in the target environment.

1.2 Cross-Machine Determinism Analysis

When evaluating cross-machine build determinism, experimental runs revealed that package sets and file lists remain 100% identical across different physical build hosts pinned to the same snapshot ID. However, minute byte-level divergences persist in specific configuration databases. For example, installing `shadow` or account-management tools updates `/etc/shadow` password modification dates based on the wall-clock days since January 1, 1970 ($d = \lfloor t_{\text{now}}/86400 \rfloor$). Similarly, database initializations (such as MariaDB in the `mysql` module) generate unique internal tablespace transaction log UUIDs.

ModFS resolves this by establishing a clear reproducibility boundary: **Package resolution, binary dependency graphs, and structural directory layouts are strictly deterministic, while runtime host identity state is isolated and regenerated during first boot.**

2 Update Strategy Costs: Intra-Generation vs. Inter-Generation

Updating software in a Bare Metal as a Service environment imposes dual overheads: server-side storage footprint and network transfer bandwidth across flashing pipelines. ModFS evaluates update economics under two distinct operational paradigms: *Intra-Generation Specification Updates* and *Inter-Generation Repository Upgrades*.

2.1 Intra-Generation Specification Updates

An intra-generation update occurs when a systems administrator modifies a module specification (e.g., adding a package to `pytools` or updating configuration parameters) while keeping the distribution snapshot pin fixed (20260901T000000Z).

Under this scenario, the modular delta structure yields massive network transfer savings. Rather than rebuilding and re-distributing a monolithic 62.8 MB root image for `webserver` or a 67.6 MB image for `pytools`, ModFS only rebuilds and transmits the specific 21.0 MB or 25.8 MB SquashFS delta archive.

Across the 31-module small adversarial cohort, the average delta size is only 7.5 MB, compared to an average monolithic image size of 49.2 MB. Intra-generation specification updates achieve a **79.8% reduction in transferred bytes** ($1 - 7.5/49.2 = 0.7987$), allowing rapid iterative deployment over low-bandwidth management networks.

2.2 Inter-Generation Repository Upgrades and Atomic Move Mechanics

An inter-generation upgrade occurs when the infrastructure operator advances the global repository snapshot pin (e.g., moving from Ubuntu Jammy snapshot 20260901T000000Z to 20261001T000000Z) to ingest security patches and upstream package revisions.

Unlike container layers that attempt dynamic cross-version rebasing, ModFS strictly enforces an **Atomic Generation Move** policy:

1. **Invalidation of Cross-Snapshot Deltas:** Package maintainer scripts, shared library symbols, and `dpkg` selection states evolve across snapshot dates. Stacking a module built against Snapshot S_1 over a Base Image built against Snapshot S_2 triggers a Class 2 Conflict (*Version Skew*). Tier 1 static admission checks detect snapshot mismatch in 100% of cases and reject the combination (**REJECT**).
2. **Complete Generation Re-Compilation:** When the repository snapshot pin is updated, the server-side pipeline executes a full generation build: the base image is re-debootstrapped and upgraded against the new snapshot, and all 40 module deltas are re-compiled in parallel.
3. **Transfer Economics of Inter-Generation Shifts:** Because `dpkg` maintainer scripts re-evaluate package state against upgraded base libraries, every delta archive undergoes byte-level changes. Consequently, inter-generation updates require re-shipping all module deltas. Space savings during inter-generation moves stem entirely from *server-side base-sharing within the new generation*, rather than incremental delta recycling across generations.

3 Base Image Sizing Trade-offs and Fleet Distribution Models

A critical architectural decision in modular filesystem design is determining the contents of the shared read-only base image (B). ModFS employs a **Minimal Base Image** philosophy (`base.sqsh`, 41.7 MB, 113 core packages), containing only essential system utilities, `systemd`, `APT/dpkg`, and basic networking tools.

To justify this choice, we analyze the trade-off against a **Fat Base Image** that pre-installs common development toolchains (`gcc`, `python3-pip`, `default-jdk-headless`, `git`).

3.1 The Fleet Distribution Formalism

Consider a bare-metal server fleet consisting of K physical bare-metal nodes deploying M distinct workload modules. Let S_B denote the base image size, and $S_\delta(m)$ denote the delta size of module m .

Under a **Minimal Base** ($B_{\min}$):

$$S_{B_{\min}} = 41.7 \text{ MB} \quad (1)$$

Workload modules that require development toolchains (e.g., `gcc`, `rust`, `java`) must bundle those packages inside their own deltas:

$$S_{\delta, \text{gcc}} = 84.0 \text{ MB}, \quad S_{\delta, \text{rust}} = 175.5 \text{ MB}, \quad S_{\delta, \text{java}} = 141.6 \text{ MB} \quad (2)$$

Under a **Fat Base** (B_{fat}) that incorporates `build-essential`, `python3`, `default-jdk-headless`, and `git`:

$$S_{B_{\text{fat}}} = 41.7 + 84.0 + 25.8 + 141.6 + 17.1 = 310.2 \text{ MB} \quad (3)$$

Under B_{fat} , the delta sizes for `gcc`, `pytools`, and `java` shrink to near zero (~ 0.5 MB).

3.2 Fleet Trade-off Analysis

The total storage/network footprint across a fleet of K nodes deploying workload distribution $P(m)$ is governed by:

$$\text{Footprint}_{\text{fleet}} = K \cdot S_B + \sum_{k=1}^K S_\delta(m_k) \quad (4)$$

- **Homogeneous Development Fleet:** If 100% of nodes (K nodes) run heavy compilation workloads utilizing `gcc`, `java`, and `python3`, the fat base eliminates duplicate transfer of toolchain packages, saving network bandwidth.
- **Heterogeneous Microservice Fleet:** In a realistic Bare Metal as a Service cloud, target nodes run specialized, single-purpose microservices. A node provisioned for `redis` (1.7 MB delta) or `memcached` (10.3 MB delta) requires only $41.7 + 1.7 = 43.4$ MB under a minimal base. Under a fat base, that same `redis` node would be forced to pull $310.2 + 0.5 = 310.7$ MB—a $7.15\times$ **storage inflation** containing 268 MB of unused compilers and runtimes.

Therefore, ModFS adopts the minimal base policy to guarantee that bare-metal nodes never ingest unused, unneeded software payloads, minimizing both storage overhead and attack surface.

4 System Boundaries and Architectural Constraints

To maintain rigorous scientific positioning, this section delineates the explicit technical boundaries and operational constraints of the ModFS architecture.

4.1 Server-Side Root Flattening vs. On-Node Dynamic Switching

ModFS is designed specifically as a **server-side build, verification, and composition framework** for Bare Metal as a Service.

- **Server-Side Flattening:** The ModFS composition engine stacks SquashFS deltas over the base image, executes semantic record-level registry reconciliation, and flattens the resulting merged directory tree into a single, contiguous **ext4** root filesystem image. This image is flashed directly onto target physical drive partitions (or served via iSCSI/NFS root).
- **No On-Node Live Module Switching:** ModFS does *not* run a dynamic runtime manager on physical nodes to unmount or insert module layers while the operating system is live. Live unmounting of lower OverlayFS layers under running processes introduces kernel memory page invalidations and file descriptor breakage. ModFS avoids runtime instability by executing all composition and reconciliation statically on the server prior to bare-metal deployment.

4.2 Flat Sibling Delta Topology Constraint

ModFS strictly enforces a **Flat Sibling Delta Topology**:

$$\text{Root View} = \text{Base} + \sum_{i=1}^N \text{Delta}_i + \text{Reconciliation Layer} \quad (5)$$

All module deltas ($M_1, M_2, \dots, M_N$) are direct sibling deltas compiled directly against the shared Base Image (B). ModFS explicitly excludes **Nested Delta-on-Delta Hierarchies** ($B + M_1 + M_{1.1} + M_{1.1.1}$).

Rationale: Allowing nested deltas introduces exponential state divergence. If Module $M_{1.1}$ is compiled against M_1 , updating M_1 invalidates all downstream child deltas. Enforcing a flat sibling structure bounds state divergence to single-level record reconciliation and ensures that any arbitrary combination of modules can be evaluated via Tier 1 static admission in $O(N^2)$ or $O(N^3)$ time.

4.3 Kernel Module ABI Dependencies and GPU Driver Boundaries

As established in Chapter 4, evaluating hardware-dependent modules such as **nvidia-driver-535** (231.6 MB) and **cuda-runtime** (680.2 MB) exposes a strict boundary between filesystem layer validity and host kernel execution:

1. **Filesystem and Boot Soundness:** Under Tier 3 QEMU UEFI boot testing, the GPU module stack (**base + nvidia-driver-535 + cuda-runtime**) boots cleanly under **systemd** in 198 seconds, achieving 0 failed units, clean **dpkg -audit**, and passing internal library probes.
2. **Kernel Module ABI Matching:** Physical insertion of the NVIDIA kernel driver (**nvidia.ko**) into a running Linux kernel requires that the target kernel ABI exactly matches the compilation kernel headers (**linux-objects-nvidia-535-5.15.0-185-generic**).
3. **UEFI Secure Boot and Signing:** Flashing modular root filesystems on bare-metal hardware equipped with UEFI Secure Boot requires that compiled kernel module deltas contain valid cryptographic signatures trusted by the host machine's Machine Owner Key (MOK) keyring.

5 Threats to Validity

Evaluating a complex filesystem architecture introduces potential threats to both internal and external validity.

5.1 Internal Validity

Internal validity addresses whether the experimental setup and measurement tools accurately reflect true system performance without introducing measurement artifacts.

- **QEMU Virtualization Overhead vs. Physical Hardware:** Tier 3 boot testing was conducted inside QEMU/KVM virtual machines with virtualized virtio storage controllers. While system startup sequence, `systemd` service ordering, and socket binding behavior are functionally identical to physical hardware, absolute boot durations (e.g., 183–330 seconds) reflect virtual disk I/O and QEMU serial console logging delays. Bare-metal NVMe flashing and local boot execution times will be significantly faster.
- **Harness Timing Variance:** Tier 2 composition timing sweeps were executed on a quiet reference host. While Ordinary Least Squares (OLS) regression models achieved high explanatory power ($R^2 = 0.9747$), background kernel VFS page cache flushing introduces minor millisecond-level noise in mount duration (t_{mount}).

5.2 External Validity

External validity addresses the extent to which the empirical results generalize to other Linux distribution ecosystems and hardware environments.

- **Debian/Ubuntu Specificity:** ModFS’s semantic reconciliation engine (`reconcile.py`) relies on specific knowledge of `dpkg` status stanza formatting, `debconf` templates, and `update-alternatives` priorities. Porting ModFS to Red Hat Enterprise Linux (RPM/`dnf`) or Alpine Linux (`apk`) would require developing distribution-specific registry parsers and maintainer script hooks, though the underlying OverlayFS/SquashFS delta composition model remains universal.
- **Evaluation Catalogue Diversity:** The 40-module catalogue spans server applications, developer toolchains, database engines, and synthetic stress cases. While representative of common enterprise cloud workloads, ultra-dense distributions containing thousands of desktop GUI packages or non-FHS proprietary software stacks may introduce novel conflict classes not observed in the 40-module evaluation set.

Chapter 6: Conclusion and Future Work

Thesis Core Body Final Chapter

ModFS: A Modular, Updateable File System for Bare Metal as a Service

September 21, 2026

Abstract

This concluding chapter synthesizes the core contributions of **ModFS**, provides evidence-based answers to the three central research questions, and outlines promising directions for future research. ModFS demonstrates that archive snapshot pinning reduces an otherwise intractable file-level conflict problem into a small, declarative, package-level verification problem, enabling high storage efficiency, deterministic builds, and verified bootability for Bare Metal as a Service (BAAS) infrastructure.

Contents

1	Summary of Core Contributions	2
2	Definitive Answers to Research Questions	2
2.1	Answer to RQ1: Conflict Classification and Reconciliation	2
2.2	Answer to RQ2: Storage Efficiency Across Workload Scales	3
2.3	Answer to RQ3: Verification and Boot Reliability	3
3	Directions for Future Work	4
4	Closing Remarks	4

1 Summary of Core Contributions

Bare Metal as a Service (BAAS) infrastructure requires rapid, predictable, and storage-efficient host provisioning. Traditional deployment pipelines rely on monolithic, whole-disk binary images, resulting in severe storage duplication, network bandwidth overhead, and high deployment latency when multiple server workloads share a common operating system baseline.

This thesis presented **ModFS**, a modular, updateable filesystem architecture designed specifically for BAAS environments. ModFS addresses these challenges through five fundamental technical contributions:

1. **Archive Snapshot Repository Pinning:** ModFS freezes package resolution against fixed archive snapshots (`snapshot.ubuntu.com/ubuntu/<TIMESTAMP>`), transforming dynamic repository resolution into a deterministic, time-invariant operation and eliminating version skew across independently constructed modules.
2. **Pinned Sibling OverlayFS Delta Squashing:** ModFS constructs per-use-case modules by installing packages into an OverlayFS view over a read-only `debootstrap` base image and squashing only the upper directory (`upperdir`) into an immutable SquashFS archive (`.sqsh`). This preserves FHS directory structure while capturing minimal per-module diffs.
3. **9-Class Conflict Taxonomy and Semantic Record-Level Reconciliation:** ModFS establishes a comprehensive 9-class conflict taxonomy covering benign overlap, version skew, declared conflicts, file collisions, state divergence, base drift, numeric identity collisions, runtime resource races, and opaque directory erasure. To resolve state divergence, ModFS implements a semantic record-merge engine (`reconcile.py`) that executes record-level union functions over seven unowned system registries (`dpkg status`, `debconf`, account databases, `extended_states`, alternatives, diversions, and linker caches) inside a top reconciliation layer.
4. **Append-Only UID/GID Range Partitioning:** ModFS assigns disjoint, 100-wide numeric identity windows (starting at 2000+) to individual module specifications, configuring build-time account allocation tools (`adduser.conf`, `login.defs`) to prevent numeric UID/GID collisions across independently built deltas.
5. **Stratified Three-Tier Evaluation Framework:** ModFS implements a rigorous validation pipeline comprising Tier 1 rootless static metadata admission, Tier 2 physical layer composition and structural assertion verification (V1–V8), and Tier 3 automated QEMU UEFI boot testing under `systemd` with guest service probe probes.

2 Definitive Answers to Research Questions

The empirical findings from the 40-module evaluation catalogue, 780 pairwise sweeps, 9,880 triplet sweeps, 152 physical composition experiments, and automated QEMU UEFI boots provide definitive answers to the three central research questions formulated in Chapter 1:

2.1 Answer to RQ1: Conflict Classification and Reconciliation

RQ1: *How can file-level and package-level state conflicts across independent filesystem deltas be systematically classified, prevented, and reconciled?*

Answer: State conflicts across independent filesystem deltas can be systematically managed by categorizing them into a 9-class conflict taxonomy and applying targeted policies at build time, admission time, and composition time:

- **Prevention at Build Time:** Class 6 Base Drift is prevented by upgrading the base image against the snapshot pin prior to module creation. Class 7 Identity Collisions are prevented by partitioning append-only 100-wide UID/GID ranges (2000+) in `specs/uid-ranges.yaml`. Class 9 Opaque Directory Erasure is prevented by stripping `trusted.overlay.opaque` extended attributes during SquashFS delta compression.
- **Admission at Pre-Flight Time:** Class 2 Version Skew and Class 3 Declared Conflicts are rejected during Tier 1 static manifest analysis. Class 4 File Collisions are detected by intersecting manifest file lists, allowing unowned path overlaps while rejecting true payload collisions unless mediated by `dpkg` diversions or `Replaces` declarations.
- **Reconciliation at Composition Time:** Class 5 State Divergence is resolved by executing record-level union functions over non-package-owned registry files (`dpkg status`, `debconf`, `passwd`, `group`, `shadow`, `gshadow`, `subuid`, `subgid`, `extended_states`, `diversions`) and regenerating derived state (`/etc/ld.so.cache` via `ldconfig` and `/etc/alternatives/*` via `update-alternatives -auto`) inside the top reconciliation layer.

2.2 Answer to RQ2: Storage Efficiency Across Workload Scales

RQ2: *What server-side storage savings does the pinned sibling delta model achieve compared to traditional monolithic disk images across diverse workload scales?*

Answer: The server-side storage efficiency of ModFS depends strongly on the software payload characteristics of the module cohort:

- **Small Adversarial / Real Application Cohort ($n = 31$):** ModFS achieves a $5.59\times$ **storage reduction** (82.1% space savings), reducing total server-side storage from 1,524.3 MB (monolithic equivalents) to 272.8 MB (ModFS base + deltas). The mean delta size in this cohort is just 7.5 MB.
- **Large Realistic Workload Cohort ($n = 9$):** For massive development toolchains and GPU drivers (`cuda-runtime` 680.2 MB, `nvidia-driver-535` 231.6 MB, `rust` 175.5 MB, `java` 141.6 MB, `llvm` 128.2 MB), the storage reduction ratio is $1.20\times$ (16.4% space savings), as huge proprietary binaries dominate the module footprint.
- **Whole 40-Module Catalogue ($n = 40$):** Across the entire evaluation catalogue, ModFS achieves an overall $1.84\times$ **storage reduction** (45.7% space savings), storing 1,935.6 MB compared to 3,562.5 MB for monolithic images.

2.3 Answer to RQ3: Verification and Boot Reliability

RQ3: *How effectively can static metadata admission and structural verification predict the bootability and runtime service health of composed modular systems?*

Answer: Stratified verification provides an exceptionally reliable predictor of image composition and runtime boot success:

- **Tier 1 Static Metadata Admission:** Fast, rootless manifest analysis filters out 100% of version-skewed or declared-conflict combinations in sub-millisecond execution times (< 1.0 ms per check), correctly rejecting positive control modules (`control-oldsnap`).
- **Tier 2 Physical Layer Verification:** Physical layer composition and structural assertion testing (V1–V8) achieved a 100% **clean pass rate** (152/152) across all stacked compositions up to $N = 38$ layers. Composition latency scales linearly according to $t_{\text{compose_total}} = 175.2 + 30.55N$ ms ($R^2 = 0.9747$).

- **Tier 3 QEMU UEFI Boot Validation:** Automated virtual machine boots under UEFI firmware and `systemd` confirm that Tier 1 and Tier 2 clean compositions reliably reach system bootability, clean `dpkg -audit` execution, and functional service listener startup. Tier 3 successfully detects Class 8 Runtime Resource Conflicts (socket binding collisions between `nginx` and `apache2` on port 80), confirming that runtime races sit strictly at the service probing boundary.

3 Directions for Future Work

While ModFS delivers a complete, verified, and performant modular filesystem pipeline, several avenues remain for future research and engineering extensions:

1. **Multi-Distribution Package Management Support:** Extending ModFS beyond Debian/Ubuntu (`dpkg/APT`) to RPM-based distributions (Red Hat Enterprise Linux, Fedora via `dnf/rpm`) and Alpine Linux (`apk`). This requires implementing distribution-specific registry parsers for RPM databases (`/var/lib/rpm/rpmdb.sqlite`) and `apk` world files.
2. **Automated Runtime Port Remapping and Socket Isolation:** Class 8 Runtime Resource Conflicts (such as port 80 collisions between web servers) currently trigger runtime service probe failures. Future work could integrate automated socket remapping rules, `systemd` socket activation overrides, or network namespace isolation wrappers during Tier 2 reconciliation to allow conflicting daemons to co-exist cleanly.
3. **Dynamic Bare-Metal Streaming and Block-Level Diffing:** Rather than flattening stacked SquashFS deltas into an `ext4` root image on the server prior to network flashing, future deployment engines could stream SquashFS deltas directly over iPXE/HTTP and assemble an in-memory OverlayFS root during early `initramfs` execution on bare-metal target nodes.
4. **Secure Boot Kernel Module Signing and Driver Integration:** Enabling full Secure Boot compliance for proprietary drivers (`nvidia-driver-535`) by automatically signing built kernel modules (`.ko`) with an infrastructure Machine Owner Key (MOK) during delta compilation.
5. **Subuid/Subgid Range Partitioning for Unprivileged Containers:** Extending the UID/GID range partitioning scheme (2000+) to support dynamic subuid and subgid allocations (`/etc/subuid`, `/etc/subgid`), enabling unprivileged rootless container execution (e.g., Podman, Docker rootless) inside composed modular environments.

4 Closing Remarks

ModFS demonstrates that combining immutable compressed archive storage (SquashFS), kernel union mounts (OverlayFS), archive snapshot repository pinning, and semantic registry reconciliation yields a highly effective, deterministic, and storage-efficient filesystem architecture for Bare Metal as a Service. By bridging the gap between declarative package metadata and physical file system state, ModFS provides a robust foundation for next-generation bare-metal cloud infrastructure.